LLMKernelBench: Benchmarking Large Language Models on Software Vulnerability Detection in Linux Kernel

Arastoo Zibaeirad, Rodrigo Pato Nogueira, and Marco Vieira

***Abstract*—Large Language Models (LLMs) demonstrate strong capabilities in code-related tasks, however their effectiveness in Software Vulnerability Detection (SVD) remains poorly understood due to inadequate evaluation frameworks. Existing benchmarks suffer from training data contamination, isolated function evaluation without cross-component context, lack of strict vulnerable-patched pairing, and binary classification without hierarchy-aware Common Weakness Enumeration (CWE) assessment, which prevents a reliable measurement of security reasoning versus pattern matching on leaked data. We introduce LLMKernelBench, a rigorous benchmark comprising 417 real-world Linux kernel vulnerabilities across 74 CWE types, split into a Primary Benchmark Dataset (PBD; 314 samples, ≤2024) and a Leakage Free Dataset (LFD; 103 samples, 2025 post-cutoff), with context-aware evaluation at three granularity levels and hierarchy-aware metrics quantifying semantic proximity in misclassifications. We evaluate seven LLMs spanning code-specialized and general-purpose architectures. Binary vulnerability detection is near-random ($\sim 50\%$ accuracy) and strongly biased: some models label $> 75\%$ of samples as vulnerable, while others mostly label them as non-vulnerable. CWE prediction is effectively unusable, with an average Top-1 accuracy of ~~1.26%~~ 1.4% (best: ~~2.92%~~3.3%) and a ~~10.59%~~ 12.3% hierarchy proximity score, ~~indicating almost no understanding of vulnerability families~~providing little reliable exact or taxonomy-level signal. On the leakage-free 2025 split, binary accuracy remains near-random and robustness to multi-file abstraction is model-specific rather than tied to specialization, with code-specialized and general-purpose models ~~degrade minimally (0.3 percentage points) , while code-specialized models drop sharply (8.8~~ degrading by 2.6% and 4.5% on average, respectively. The micro-to-macro CWE-accuracy gap is larger on LFD (6.6 points) than on PBD (0.9 points), ~~suggesting that code-specialized models exhibit behavior consistent with memorization-based pattern matching rather than robust security reasoning~~which is consistent with sensitivity to class frequency but does not identify an internal model mechanism. Code and data are available at https://anonymous.4open.science/r/LLMKernelBench-785B/.**

***Index Terms*—Large Language Models, Software Security, Software Vulnerability Detection**

I. Introduction

Software vulnerabilities pose a critical threat to modern computing infrastructure. The number of identified CVEs ~~has~~ surged to over 29,000 in 2023, up from 20,153 in 2021 [?], underscoring the urgent need for automated Software Vulnerability Detection (SVD). Real-world vulnerabilities rarely ~~manifest~~ occur in isolated code fragments~~but instead involve~~ , instead involving intricate interactions across ~~multiple~~ functions, files, and modules. Detecting ~~these~~ them requires not only identifying ~~whether code is vulnerable~~ vulnerable code but also classifying the specific Common Weakness Enumeration (CWE) category~~(~~, which is essential for triage, prioritization, and ~~linking to mitigationstrategies)~~mitigation. As production systems such as the Linux kernel (∼30M LOC) power Android, servers, and embedded devices, and because diverse, high-impact flaws persist, effective automated detection is critical for software security at scale.

Traditional SVD approaches include Automated Program Repair, fuzzing, and Static Analysis Tools (SATs), but these face ~~fundamental~~ key limitations: patch quality issues [?], [?], coverage gaps and logic error blindness [?], [?], and high false positive rates [?]. Recent work has turned to Large Language Models (LLMs) [?], [?], [?], [?], [?], which demonstrate strong capabilities in code understanding, test generation [?], [?], and various programming tasks [?]. Multiple benchmarks have emerged to evaluate LLM security reasoning, incorporating vulnerability-specific metadata [?], [?], [?] and CWE-wise analysis [?], [?].

Despite progress, existing benchmarks exhibit critical limitations. First, most rely on historical data that likely overlaps with LLM training corpora, artificially inflating metrics through *training data contamination*, with only limited efforts [?] addressing this via post-cutoff validation. Second, datasets often extract *isolated functions* without surrounding context (macros, type definitions, cross-file dependencies), failing to capture how real vulnerabilities ~~manifest~~ emerge through component interactions. Third, many lack *strict vulnerable-patched pairing*, instead collecting functions ~~across~~ from different commits or versions, introducing label noise [?], [?], [?]. Fourth, evaluations mainly focus on binary classification~~without assessing~~ , overlooking *CWE hierarchy understanding* ~~(whether models grasp~~ and whether models capture vulnerability families or ~~merely memorize surface patterns)~~simply memorize patterns. These gaps ~~prevent a~~ hinder reliable assessment of whether LLMs ~~possess~~ exhibit genuine security reasoning ~~versus~~ or pattern matching on contaminated data.

In this paper, we present LLMKernelBench, a rigorous benchmark addressing these limitations through three key design principles. First, we introduce a *temporal split*: the Primary Benchmark Dataset (PBD) covers vulnerabilities through 2024, while the Leakage Free Dataset (LFD) contains only 2025 post-cutoff CVEs, enabling ~~direct measurement of~~

Arastoo Zibaeirad, Rodrigo Pato Nogueira, and Marco Vieira are with the University of North Carolina at Charlotte, Charlotte, NC, USA (e-mail: azibaeir@charlotte.edu; rpatodec@charlotte.edu; marco.vieira@charlotte.edu).

~~generalization versus memorization~~comparison of historical and post-cutoff performance while reducing direct exposure risk in LFD. Second, we adopt a *context-aware* approach, pairing 417 real-world Linux kernel CVEs with strict vulnerable-patched pairing while preserving surrounding non-functional elements that influence vulnerability semantics. Third, we evaluate across three *abstraction levels* (single function, multiple functions, multiple files) to assess context utilization, which is essential for deployment but has not been explored in prior work. We evaluate seven pre-trained LLMs spanning code-specialized (CodeLlama, StarCoder2, ~~Qwen2.5-Coder~~Qwen3-Coder) and general-purpose models (Llama3.1, Mistral, DeepSeek-R1, GPT-4.1-mini) across 74 CWE types, measuring both binary detection and hierarchy-aware CWE classification through novel metrics (Hierarchical Proximity Score (HPS)) that reward ~~semantic~~ taxonomic proximity.

Our work intentionally targets the Linux kernel as a representative example of a *high-assurance, security-critical software system*. The kernel's extensive use of macros, low-level memory management, and long-evolving coding conventions reflects the complexity of infrastructure software, where vulnerabilities have severe consequences. Accordingly, LLMKernelBench is not designed to characterize LLM performance on general-purpose application code, but rather to stress-test vulnerability-detection capabilities under conditions representative of real-world, safety- and security-critical systems.

Our evaluation addresses four research questions:

- **RQ1: Binary SVD performance**: How effectively can large language models detect whether a code sample is vulnerable or non-vulnerable?
- **RQ2: Consistency across CWEs pillars**: Are model performances consistent across different CWEs pillars?
- **RQ3: Consistency across granularity levels**: Are model performances consistent across different code granularities (file-, function-, and multi-file-level)?
- **RQ4: ~~Vulnerability type identification~~CWE ranking performance**: When a vulnerability is detected, how accurately can models identify its corresponding CWE category?

Contributions. In summary, this work offers:

- **Leakage-aware benchmark with temporal split.** We introduce ~~LLMKernelBench, the first~~ a kernel-focused benchmark with ~~systematic~~ explicit temporal contamination control, including 417 real-world Linux kernel CVEs split into PBD (314 samples, $\leq$2024) and LFD (103 samples, 2025 post-cutoff), enabling ~~direct measurement of generalization versus memorization~~comparison under substantially reduced direct-exposure risk.
- **Context-aware evaluation at three abstraction levels.** Unlike prior work using isolated functions, we provide strict vulnerable-patched pairing with surrounding context (macros, type definitions, globals) and evaluate at L1 (single function), L2 (multiple functions), and L3 (multiple files) to assess whether models leverage cross-component dependencies.
- **Hierarchy-aware CWE evaluation metric.** We develop the HPS metric combining Top-k accuracy, Mean Reciprocal Rank (MRR), and CWE taxonomy structure to quantify ~~semantic~~ taxonomic proximity in misclassifications, distinguishing ~~whether models confuse similar weakness families or make random errors~~ nearby hierarchy errors from unrelated labels.
- **Comprehensive evaluation of seven LLMs.** We evaluate code-specialized (CodeLlama, StarCoder2, ~~Qwen2.5-Coder~~Qwen3-Coder) and general-purpose models (Llama3.1, Mistral, DeepSeek-R1, GPT-4.1-mini) across 74 CWE types. We focus on open-source models to ensure transparency, reproducibility, and cost-effectiveness.

From a reliability engineering perspective, vulnerability detection models act as *diagnostic components* within a broader software assurance pipeline. Their practical value depends not only on average accuracy, but also on predictable behavior under realistic operating conditions, robustness to unseen inputs, and stable failure modes when confidence is unwarranted. Our ~~results show that current LLM-based approaches fail to~~ evaluated zero-shot models do not meet these reliability requirements: they exhibit near-random decision accuracy, ~~extreme~~ strong class bias, ~~high variance~~ and substantial between-model variation across abstraction levels~~, and sharp performance degradation on leakage-free data. These behaviors indicate that, in their current form, LLMs cannot be relied upon as dependable~~. These observed behaviors make them unsuitable as standalone vulnerability detectors in safety- or security-critical systems. By ~~systematically~~ characterizing these failure modes under controlled, leakage-aware conditions, this work provides ~~empirical evidence for understanding~~ evidence about the reliability limits of ~~LLM-based analysis tools and serves as~~ baseline LLM-assisted analysis and a benchmark for assessing future improvements~~in dependable AI-assisted software assurance~~.

The remainder of this paper is structured as follows: **§II** reviews background and related work. **§III** presents the benchmark and evaluation protocol. **§IV** reports findings for RQ1–RQ4. **§V** discusses implications and limitations. **§VI** summarizes threats to validity. **§VII** concludes the paper.

II. Background and Related Work

CWE Taxonomy and Hierarchical Evaluation: The CWE system [**?**] organizes software weaknesses into a four-level hierarchy: Pillars, Classes, Base weaknesses, and Variants. This enables graduated assessment, where identifying a parent category ~~demonstrates partial understanding~~ receives partial taxonomic credit even when the specific variant is misidentified. Our work leverages this hierarchy to measure both exact classification accuracy and ~~semantic~~ taxonomic proximity through the HPS metric.

Vulnerability Datasets – From File-Level to Context-Aware: Early datasets adopted *file-level* labeling, marking entire files touched by security commits as vulnerable [**?**], [**?**], [**?**], [**?**]. While enabling large-scale collection, this introduced label noise, class imbalance [**?**], and difficulty distinguishing security fixes from refactoring [**?**], [**?**]. The field shifted toward *function-level* granularity [**?**], [**?**], but these often failed to

TABLE I
VD BENCHMARKS AT A GLANCE

Benchmark	Scale	CWE	Pairs (vuln vs patch)	Leakage	Granularity
[?] (2019)	27,318	–	✗	✗	Func
[?] (2022)	130	75	✓	✗	Snippet
[?] (2024)	1000	48	✓	✗	Program
[?] (2025)	294	77/86	✓	partial	Func.
[?] (2025)	5,000	25	partial	✗	Func.+project
[?] (2024)	228	8	✓	✓	Func.
LLMKernelBench	417	74	✓	✓	Func.+context

strictly pair vulnerable-patched versions of the *same* function or account for cross-component dependencies.

Unlike prior work extracting isolated functions, LLMKernelBench adopts a *context-aware* approach, pairing line-modified functions with surrounding non-functional elements (macros, type definitions, globals) that influence vulnerability semantics. By evaluating at three abstraction levels (single function, multiple functions, multiple files), we assess whether LLMs can leverage broader context, essential for deployment but unexplored in existing benchmarks.

LLM Benchmarks – From Scale to Quality and Leakage Control: Early benchmarks prioritized *scale*, collecting tens of thousands of samples but lacking vulnerability-specific metadata [**?**], [**?**]. The field evolved toward richer annotations [**?**], [**?**], [**?**] and CWE-wise analysis [**?**], [**?**], yet a critical gap remains: *training data contamination*. Most benchmarks rely on historical data that likely overlaps with model training corpora, artificially inflating metrics. Only limited efforts [**?**] have considered post-cutoff validation, yet none have used systematic temporal splits. Recent supervised detectors report substantially higher aggregate accuracy under different assumptions. For example, SecureFalcon fine-tunes a Falcon-derived classifier on FormAI/FalconVulnDB and reports 94% binary and 92% multiclass accuracy for C/C++ vulnerability classification [**?**], while Alam et al. [**?**] fine-tuned GPT-3.5 Turbo and GPT-4o Mini and reported 99% vulnerability-detection accuracy on Solidity. These results demonstrate the value of task-specific fine-tuning and large task-specific datasets, but they are not directly comparable to our zero-shot evaluation of off-the-shelf LLMs on temporally separated Linux kernel CVEs.

LLMKernelBench addresses these limitations by prioritizing *quality over scale* and *leakage-aware* evaluation (Table I). We provide 314+103 curated CVE-linked samples across 74 CWE types with strict vulnerable-patched pairing. Critically, we introduce a temporal split: PBD (covering CVEs through 2024) versus LFD (covering 2025 post-training-cutoff CVEs), enabling ~~direct measurement of generalizationversus memorization~~a historical-versus-post-cutoff comparison rather than treating historical performance alone as evidence of generalization. Our hierarchy-aware evaluation and context-aware abstraction levels position LLMKernelBench as a *stress-test benchmark* for security ~~reasoning~~analysis, complementing broader training corpora while addressing systematic limitations in rigor and contamination control.

III. BENCHMARKING APPROACH

LLMKernelBench is designed to assess pre-trained LLMs (see Table IV for the models evaluated) on SVD and CWE-label classification tasks, focusing on C code from the Linux kernel while being extensible to other programming languages. As illustrated in Figure 1, the framework is based on vulnerability metadata and code pairs extracted from Linux CVE commits, and includes prompts designed for SVD and CWE-label classification tasks. ~~LLMKernelBench evaluates the models in a zero-shot setting with temperature set to~~ Raw model outputs are processed through the LangChain `VulnerabilityOutputParser`; for response-validity analysis, we additionally audit the raw text to distinguish genuine uncertainty from malformed or otherwise non-compliant output. The primary open-source SVD evaluation uses temperature 0 ~~to ensure deterministic and reproducible outputs, using~~ for deterministic decoding. The response-validity sweep averages the two complete temperature-0.1 runs, including GPT-4.1-mini, whereas the CWE-ranking analysis averages its three available ranking runs. We use binary classification metrics (precision, recall, accuracy, F1) for SVD and ranking metrics for CWE-label classification. This approach ensures benchmark reliability through representativeness (diverse real-world vulnerabilities), adaptability (an extendable methodology), portability (applicability across various LLMs), and fairness (standardized evaluation procedures).

Section III-A details our dataset construction process. Section III-C defines metrics for ~~measuring effectiveness in~~ both SVD and CWE-label classification tasks. Finally, Section III-D presents our standardized prompt templates.

A. Dataset

Our method systematically extracts pairs of vulnerable code blocks and their corresponding patched versions from real-world Linux kernel commits. We focus on the Linux kernel due to its rich vulnerability history and critical security importance. We collect commit hashes, CVE identifiers, and CWE classifications from well-known vulnerability databases, including NVD [**?**], in line with established practices [**?**]. For each vulnerability-fixing commit, we retrieve the vulnerable code from its parent commit (pre-fix version) and the patched code from the commit itself (post-fix version).

Our extraction process identifies all security-related functions and non-functional elements (global variables, macros, type definitions, structs/unions, headers) that were directly modified in each commit. We include only functions with actual line-level changes, excluding unchanged code to avoid label noise. Non-functional elements are tracked separately as they often form critical vulnerability contexts that influence execution across multiple functions. This comprehensive approach assembles cohesive, vulnerable, and patched code blocks at multiple abstraction levels (single function, multiple functions, multiple files), enabling LLMs to analyze complete contextual information rather than isolated fragments.

After extraction, the dataset is divided into two. The PBD covers all vulnerabilities discovered on or before 2024, while the LFD covers the more recent 2025 vulnerabilities. This division enables us to evaluate the impact of potential data leakage on the model's performance, thereby increasing our confidence in the generalizability of our findings.

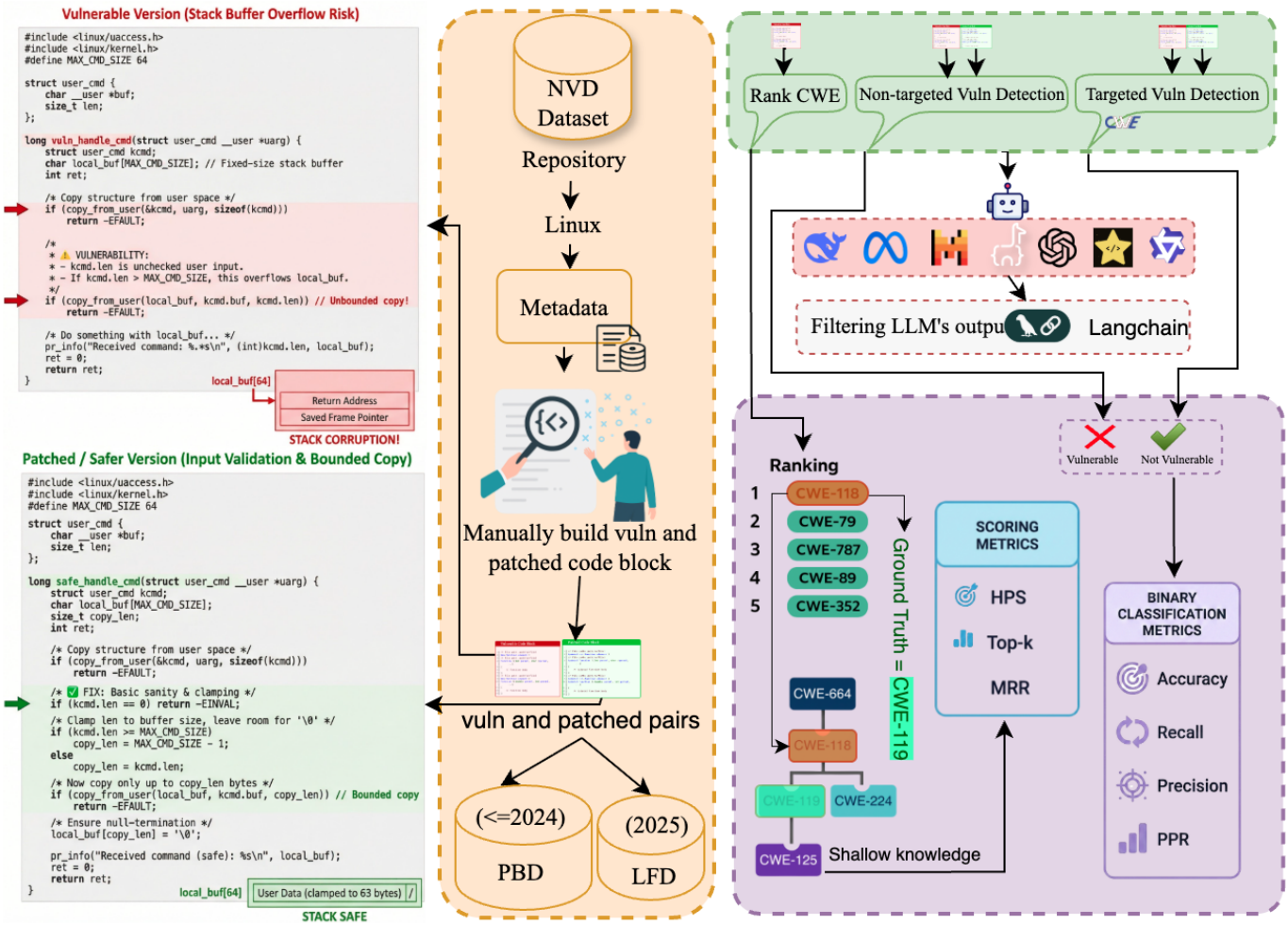

Fig. 1. LLMKernelBench Framework. Metadata (commit messages, diffs, function/file names) and vulnerable-patched code pairs are extracted from Linux CVE commits. Prompts are designed for SVD and CWE-label classification tasks in zero-shot settings. LLMs ~~performs~~ perform detection and classification~~. Outputs are filtered via regex. Detection is evaluated using~~ , while LangChain parses raw model outputs and a response audit separates binary verdicts, genuine abstentions, and ~~ranking metrics.~~invalid outputs
. Detection is evaluated using binary and ranking metrics.

Manual Review, Denoising, and Deduplication. Our automated extraction process allowed us to collect a total of 400 samples for PBD and 110 samples for LFD. After that, to ensure dataset integrity and label fidelity, both datasets were manually validated by two different reviewers. For this validation, a standardized protocol that defined inclusion criteria, exclusion criteria, and required evidence documentation was first defined. Then, the original datasets were split in half, and each half was assigned to a different reviewer. The reviewers followed the previously defined documentation to clean the dataset. Doubtful cases were marked for discussion and resolved through consensus sessions between the reviewers. The following validations were performed:

- **Denoising**: Removed samples and functions/files where the changes were not security-related.
- **Completeness check**: Ensured all relevant and security-impacting changes from each commit were included.
- **CWE frequency filtering**: CWEs that appeared only once were removed to improve representativeness.
- **Comment removal**: Stripped all code comments from both vulnerable and patched code blocks to eliminate potential biases and ensure that the LLMs relied solely on code structure and semantics for SVD.
- **Deduplication**: Removed duplicate or near-duplicate code samples (e.g., identical functions, commits, or cloned files) to prevent models from seeing the same or highly similar code during testing.
- **CWE labeling**: In the LFD, 40 samples lacked CWE identifiers in the NVD database; we manually labeled these samples, strictly adhering to NVD criteria for consistent labeling.

Table II summarizes the overall size and composition of the PBD and LFD after this comprehensive cleaning process. The final PBD dataset comprises ~~a total of~~ 314 samples ~~and spans~~ spanning 74 ~~different~~ distinct CWEs, while the final LFD dataset contains 103 samples belonging to 19 ~~different~~ CWEs.

CWE Distribution and Label Space. We did not pre-select a preferred subset of CWEs. PBD retains the CWE labels associated with the sampled Linux-kernel CVEs, while 40 LFD samples lacking NVD CWE identifiers were manually labeled under NVD-aligned criteria. The resulting label space,

TABLE II
LLMKERNELBENCH DATASET STATISTICS

Metric	PBD	LFD
Vulnerabilities	314	103
Unique CWEs	74	19
Files/Functions Changed (avg)	1.5 / 2.0	1.1 / 1.3
Vuln/Patched Lines (avg)	109.4 / 114.1	60.9 / 63.6
Lines Added/Deleted (avg)	17.6 / 9.0	7.0 / 3.5

TABLE III
CWE PILLAR DISTRIBUTION ACROSS PBD AND LFD.

CWE-ID	Pillar Name	Coverage (%)	
		PBD	LFD
CWE-664	Improper Control of Resource Lifetime	136 (43.3)	62 (60.2)
CWE-OTHER	Other/Unmapped CWEs	38 (12.1)	0 (0.0)
CWE-682	Incorrect Calculation	32 (10.2)	7 (6.8)
CWE-284	Improper Access Control	28 (8.9)	0 (0.0)
CWE-691	Insufficient Control Flow Management	27 (8.6)	11 (10.7)
CWE-707	Improper Neutralization	18 (5.7)	9 (8.7)
CWE-693	Protection Mechanism Failure	16 (5.1)	0 (0.0)
CWE-710	Improper Adherence to Coding Standards	11 (3.5)	14 (13.6)
CWE-703	Improper Check/Handling of Except. Conditions	9 (2.9)	0 (0.0)
CWE-435	Improper Interaction Between Components	0 (0.0)	0 (0.0)
CWE-697	Incorrect Comparison	0 (0.0)	0 (0.0)

spanning 74 distinct CWEs in PBD and 19 in LFD, therefore reflects the sampled kernel vulnerabilities rather than an artificially balanced category design. Our dataset spans diverse CWE categories organized according to the CWE-1000 hierarchical view [**?**]. Table III ~~shows the distribution of vulnerabilities across the~~ reports coverage against the ten fundamental CWE pillars~~in PBD~~: eight have nonzero coverage in the sampled data, while deprecated or otherwise unmapped labels are grouped as CWE-OTHER. CWE-664 accounts for 43.3% of PBD coverage, reflecting the prevalence of memory-related vulnerabilities in the Linux kernel. The distribution also includes 12.1% deprecated CWE identifiers (CWE-OTHER), representing historical vulnerability classifications that MITRE now marks as "PROHIBITED" for mapping. ~~This diverse distribution ensures comprehensive evaluation across different security weakness categories, from common memory safety issues to rare configuration vulnerabilities.~~

B. Tasks

Based on this dataset, the models are evaluated across two different tasks: SVD and vulnerability type identification.

SVD. In this task, the models are given either a vulnerable code block (pre-fix versions from parent commits) or a non-vulnerable (patched) code block (post-fix versions from fixing commits). They are then asked to classify the code block as vulnerable (~~1~~1), non-vulnerable (~~0~~0), or to ~~abstain from classifying it (-1).~~ answer `not sure`. The uncertainty response is represented as -1 in the structured evaluation data. This third option is used because: (1) real-world SVD often involves ambiguous cases where even security experts disagree; (2) it prevents LLMs from making forced binary decisions when evidence is insufficient, potentially reducing false positives and negatives; and (3) it mirrors practical security analysis workflows where analysts may need to flag code for further investigation rather than making definitive vulnerability determinations.

CWE-label classification. The CWE taxonomy organizes weaknesses into a hierarchical structure with four abstraction levels as shown in Figure 2: *Pillars* (highest level, 10 fundamental categories representing root security principles such as CWE-664 Improper Control of a Resource Through its Lifetime), *Classes* (general weakness types like CWE-119 Buffer Errors), *Bases* (specific weakness variants such as CWE-120 Buffer Copy without Checking Size of Input), and *Variants* (detailed platform-specific manifestations). This hierarchy is formalized in the CWE-1000 Research Concepts view [**?**].

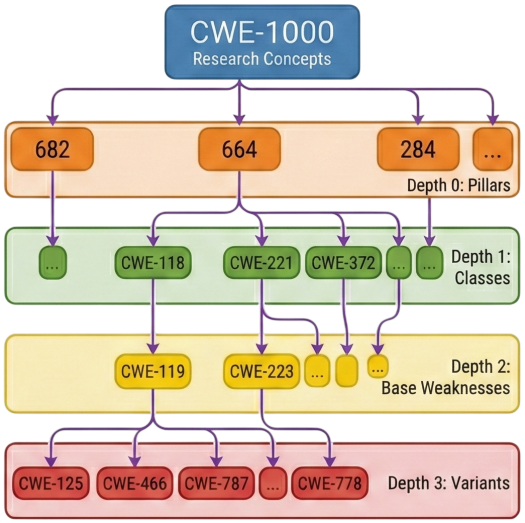

Fig. 2. CWE hierarchy.

For our CWE classification tasks, we employ a *hierarchical evaluation approach* that assesses model predictions across all ~~abstraction~~ taxonomy levels rather than focusing solely on exact matches. This design ~~enables us to measure the *depth of security knowledge* exhibited by LLMs, distinguishing between models that demonstrate only shallow,~~ measures the taxonomic proximity of model outputs, distinguishing broad category-level ~~understanding (e.g., correctly identifying a vulnerability as but unable to specify the exact type) versus those with deep,~~ matches from exact fine-grained ~~comprehension (e.g., pinpointing CWE-772 Missing Release of Resource after Effective Lifetime)~~identification without assuming that proximity alone demonstrates semantic understanding.

We introduce the HPS metric (§III-C), which ~~quantifies how hierarchically close a predicted CWE is to the true label by computing path distance~~ assigns ordinal credit according to exact, immediate parent/child, sibling, same-root, or unrelated relations in the CWE-1000 ~~taxonomy tree~~hierarchy. A prediction of ~~CWE-119 (Buffer Errors, Class level~~CWE-121 (Stack-based Buffer Overflow) when the true answer is ~~CWE-121 (Stack-based Buffer Overflow, Base level~~CWE-787 (Out-of-bounds Write) receives partial ~~credit, as it demonstrates understanding of the vulnerability family~~ child credit, recording a related candidate despite missing the ~~specific subtype~~exact type. This approach addresses practical challenges in CWE classification: fine-grained labels

(bases and variants) exhibit severe class imbalance, with many specific types having fewer than 10 examples in real-world datasets, rendering exact-match accuracy unreliable. Meanwhile, coarse-grained evaluation at only the pillar level (10 categories) would be too lenient ~~, failing to distinguish between models that truly understand security concepts versus those that merely memorize broad categories~~to distinguish precise weakness identification from broad category prediction.

Our hierarchical scoring thus ~~captures a spectrum of understanding: models achieving high~~ separates exact Top-1 ~~accuracy (i.e., the model's first prediction is the correct label; defined in §III-C2) demonstrate precise knowledge of specific vulnerability types, while those with high HPS but~~ identification from taxonomically related candidate output. High HPS with low Top-1 accuracy ~~reveal~~ indicates related labels in the candidate set, not necessarily taxonomy-aware reasoning~~, correctly identifying the vulnerability family but struggling with fine-grained distinctions~~. This multi-level evaluation ~~provides actionable insights for both model developers (to identify where models need improvement) and security practitioners (to understand whether a model's predictions are reliable enough for specific use cases)~~helps model developers locate error patterns and helps practitioners judge whether candidate labels are suitable only for routing or are accurate enough for diagnosis.

By considering both these tasks, our benchmark systematically evaluates LLMs in terms of their ability to (1) detect the presence of vulnerabilities through binary SVD classification, and (2) identify and prioritize the most likely underlying vulnerability types through CWE-label ranking.

C. Metrics

Due to the different nature of the two tasks, different metrics must be employed. These metrics, along with statistical significance testing, are discussed next.

1) SVD: To evaluate the models' performance at SVD, we consider the following terms: True Positive (TP) refers to the number of vulnerable code samples that the model correctly identifies as vulnerable; False Positive (FP) is the number of non-vulnerable samples incorrectly predicted as vulnerable; True Negative (TN) represents the number of non-vulnerable samples correctly classified as non-vulnerable; while False Negative (FN) represents the number of vulnerable samples that the model fails to identify, incorrectly classifying them as non-vulnerable. These four quantities form the basis for calculating the performance metrics described below.

It is important to note that these metrics are computed only for cases in which the models ~~actually provided a~~ provided a valid binary classification, i.e., ~~when they predicted either vulnerable (1~~vulnerable (`1`) or non-vulnerable (~~0~~`0`)~~. Samples for which the models abstained from answering (returning −1) are excludedfrom these metric calculations. In addition to reporting the standard metrics, we also provide a breakdown of how frequently each model responded versus abstained. This distinction is critical for a fair interpretation of the results (e.g., a high recall when a model only classified 5% of the samples does not carry the same significance as when it classifies 90% of the inputs).~~ `0`). Genuine abstentions and invalid outputs are excluded. We distinguish these outcomes by auditing the raw response: a *genuine abstention* clearly selects `not sure` (either alone or as an unambiguous final verdict motivated by insufficient information), whereas an *invalid response* does not yield a single resolvable verdict, for example because it continues the input code, loops, or presents contradictory choices. Reporting these categories separately is critical because parser failure or prompt non-compliance should not be interpreted as model uncertainty. The following metrics are used:

- **Accuracy**: measures the proportion of correctly classified code samples among all samples:
$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN} \tag{1}$$
As the dataset is relatively balanced between vulnerable and non-vulnerable samples, the accuracy accurately reflects the model's overall performance without being dominated by one class. High accuracy is important as it indicates that the model performs well across both classes.
- **Recall** – measures the proportion of actual vulnerable code samples that were correctly identified by the model:
$$\text{Recall} = \frac{TP}{TP + FN} \tag{2}$$
In this context, it represents the number of true vulnerabilities the LLM successfully detects or flags. High recall is crucial, as missing vulnerable samples could lead to the overlooking of potential security issues.
- **Precision** – measures the proportion of code samples predicted as vulnerable that are truly vulnerable:
$$\text{Precision} = \frac{TP}{TP + FP} \tag{3}$$
This metric indicates how many of the flagged samples are genuine vulnerabilities rather than false positives. High precision is important to reduce the number of incorrect alerts, which can waste time during manual inspection or downstream analysis.
- **Positive Prediction Rate (PPR)** – measures the proportion of all samples that the model classifies as vulnerable, regardless of correctness:
$$\text{PPR} = \frac{TP + FP}{TP + FP + TN + FN} \tag{4}$$
While this metric is not as critical as precision or recall, it provides valuable insight into the model's potential bias toward one class. A higher PPR may indicate a tendency to over-predict vulnerabilities, whereas a lower PPR suggests under-prediction. Comparing the PPR across models helps identify whether a model systematically favors one class over the other, offering additional context for interpreting performance results.

Together, these metrics provide a comprehensive evaluation of classification performance, balancing correctness, sensitivity, and class bias.

2) CWE-label classification: For the task of CWE-label classification, the following metrics are employed:

- **Top-k Accuracy** [**?**]: Considers a prediction correct if the true vulnerability type appears among the model's k

highest-ranked predictions. We measure Top-1 to Top-5 accuracy to evaluate how well models detect and rank both CVEs and CWEs. This progressive evaluation is particularly valuable in security contexts where identifying the correct vulnerability category among top candidates enables more efficient remediation efforts, even when exact classification is challenging.

- **MRR** [**?**]: Evaluates ranking quality by calculating:

$$\frac{1}{N}\sum_{i=1}^{N}\frac{1}{\text{rank}_i} \tag{5}$$

Where rank_i is the position of the correct answer in the ranked list for the i-th query. MRR rewards models that rank the correct vulnerability type higher, recognizing that, in security contexts, prioritizing findings is crucial.

- **HPS** scores a model's outputs by how close they are to the ground-truth CWE in the hierarchy. We assign weights: exact = 1.0, parent = 0.8, child = 0.7, sibling = 0.6, same-root = 0.4, unrelated = 0.0. CWE-1000 is a directed acyclic graph rather than a strict tree because a weakness may have multiple parents or occur under multiple pillars. Parent and child therefore denote any immediate taxonomy edge, siblings share at least one immediate parent, and same-root labels share at least one top-level pillar. The HPS weighting scheme follows an ordinal decay aligned with the CWE hierarchy rather than modeling precise semantic distances. Higher weights reflect closer ancestry in the taxonomy, as confusing a vulnerability with a an immediate parent or child category indicates greater understanding taxonomic proximity than predicting an unrelated weakness. The specific values are intended to preserve hierarchical ordering and distinguish near-misses from random errors, not to encode exact unrelated predictions; they do not encode calibrated semantic margins. Formally, let y_i be the ground truth and $(\hat{y}_{i1},\ldots,\hat{y}_{ik})$ the top-k predictionsY_i be the set of ground-truth labels and $(\hat{y}_{i1},\ldots,\hat{y}_{ik_i})$ the valid predicted labels. Define $r(\hat{y},y)\in\{\text{exact},\text{parent},\text{child},\text{sibling},\text{root},\text{none}\}$ and a weight map $w(\cdot)$ as above. The per-sample score is Each predicted label receives its best score against the ground-truth set,

$$\text{HPS}_i s_{ij} = \max_{1\le j\le k}\max_{y\in Y_i} w\big(r(\hat{y}_{ij},y_{\bar{i}})\big), \tag{6}$$

and the dataset score is $\text{HPS}=\frac{1}{N}\sum_{i=1}^{N}\text{HPS}_i$. HPS credits near-misses that are semantically related, which is useful for prioritization in security triageimplemented dataset score averages across all valid predicted labels,

$$\text{HPS}=\frac{\sum_i\sum_{j=1}^{k_i}s_{ij}}{\sum_i k_i}. \tag{7}$$

Thus HPS measures taxonomic proximity of the candidate set, while Top-k accuracy and MRR separately measure exact inclusion and rank. For the supporting distance audit in Tables XV and XVIII, distance is the shortest path in the undirected CWE-1000 parent–child graph. We connect the ten top-level pillars through a virtual root so that cross-pillar errors have finite distances; unmapped or deprecated identifiers remain unmeasurable. The committed HPS and distance-analysis scripts preserve all parent and pillar memberships and regenerate the reported values directly from the three ranking runs.

To ground this metric in a practical security context, consider a true CWE-787 (Out-of-bounds Write) with a predicted CWE-121 (Stack-based Buffer Overflow). CWE-121 is an immediate child of CWE-787 in the benchmark hierarchy and therefore receives the child weight of 0.7. By contrast, CWE-284 (Improper Access Control) is in a different root branch and receives 0.0. In SOC triage, the former can still route an alert toward buffer-handling review, but it must not be treated as an exact diagnosis.

By combining these metrics, we obtain a holistic view of model performance, capturing not only classification accuracyand ranking qualitybut also the semantic evaluate classification accuracy, ranking quality, and the taxonomic proximity of predicted vulnerability categories.

3) Statistical Significance Testing: To determine whether observed performance differences across experimental conditions assess whether performance differences are statistically meaningful or merely due to randomvariation, we employ three complementary statistical approaches: statistical tests for rather than random, we combine group-level differencestests, degradation metricsto quantify performance changes across abstraction levels, and Pearson correlation to assess relationships between continuous variables (e.g., context window size and model degradation), and correlation analysis.

Statistical Test Selection: To compare model performance across three test selection. To compare accuracy across abstraction levels (L1, L2, L3L1–L3), we follow a systematic approach to select the appropriate statistical test. First, we validate normality using the first check normality (Shapiro-Wilk [**?**] test for each group. Second, we test [**?**]) and homogeneity of variance using Levene's test. If both assumptions hold, we use (Levene). We then apply one-way ANOVA(parametric) [**?**] ; if only normality holds but variances differ, we use [**?**] when both hold, Welch's ANOVA ; if normality fails, we use the when variances differ, and the non-parametric Kruskal-Wallis H-test (non-parametric alternative). For PBD , all groups pass normality (Shapiro-Wilk $p>0.05$) and equal variance tests (when normality fails. PBD satisfies both assumptions (Levene $p=0.812$), justifying one-way ANOVA. For LFD, Level ANOVA, whereas LFD Level 3 fails normality ($p=0.008$) , requiring the and uses Kruskal-Wallistest. Both tests evaluate whether abstraction level significantly affects accuracy, with ; $p<0.05$ indicating significant differences and $p\ge0.05$ suggesting no meaningful effect beyond random variationdenotes a significant effect of abstraction level on accuracy.

Degradation : Measures the change in model accuracy across abstraction levels, calculated as $\text{Degradation}=\text{Accuracy}_{\text{L1}}-\text{Accuracy}_{\text{L3}}$. Positive degradation values indicate performance decline as code

scopeincreases (higher abstraction levels), while negative values indicate improvementwith broader context. This metric quantifies model robustness to code abstraction: models with quantifies robustness to context as $\text{Accuracy}_{L1} - \text{Accuracy}_{L3}$: positive values denote decline with broader scope, negative values improvement, and near-zero degradation maintain stable performance regardless of abstraction level, while high degradation reveals fragility when processing multi-file scenarios versus isolated functionsvalues stable behavior across levels.

Pearson Correlationcorrelation : Measures ($r \in [-1, 1]$) measures the strength and direction of linear relationships between two the linear relationship between two continuous variables (e.g., context window size and model degradation). The correlation coefficient (r) ranges from -1 to +1, where values close to 0 indicate no linear relationship, while values near ± 1 indicate strong positive or negative correlation. Statistical significance is assessed via vs. degradation), with significance assessed via its p-value, where $p<0.05$ indicates the correlation is unlikely due to chance.

Mann-Whitney UTest [?] : A is a non-parametricstatistical test used to compare , rank-based test for two independent groups when data do not meet the assumptions required for parametric tests (e.g., t-test). Unlike the t-test, which assumes normally distributed data with equal variances, the Mann-Whitney U test operates on ranked data and that is robust to outliers and skewed distributions. The test works by ranking all observations from both groups together, then comparing the sum of ranks between groups. The U statistic represents the number of times observations from one group precede observations from the other group in the ranking. For our abstraction level analysis, we use Mann-Whitney U or unequal-sized samples. We use it to compare Simple (L1) versus Complex (L2+L3) categories accuracy in LFD, where the small sample size for L3 sample (n=3) violates normality assumptions and creates highly unequal group sizes and group imbalance (85 vs. 18) . Statistical significance is assessed via p-value, where $p<0.05$ indicates that the observed difference between groups is unlikely due to chance alone.

violate parametric assumptions. We pair it with the **Rank-Biserial Correlationrank-biserial correlation** [?] : An effect sizemeasure for the Mann-Whitney U test that quantifies the magnitude of difference between two groups, independent of sample size. While the p-value from Mann-Whitney U tells us whether a difference is *statistically significant*effect size, rank-biserial correlation tells us whether that difference is *practically meaningful*. The rank-biserial correlation (r) is calculated as:

$$r = 1 - \frac{2U}{n_1 \cdot n_2}, \tag{8}$$

where U is the Mann-Whitney U statistic , n_1 is the size of the first group, and n_2 is the size of the second group . The correlation ranges from -1 to +1, where $r = 1$ means all observations in group 1 rank higher than all in group 2, $r = 0$ indicates no systematic difference, and $r = -1$ means all observations in group 2 rank higher. We interpret effect sizesusing conventional statistic and n_1, n_2 the group sizes, to report practical magnitude alongside significance

```
Check if the following C code block is
vulnerable.
Respond with ONLY one of these options:
- '1' if the code is vulnerable
- '0' if the code is not vulnerable
- 'not sure' if you are not sure if it is
vulnerable or not
Do not include any explanation or additional
text.
Code block:
{code}
Response:
```

Fig. 3. Vulnerability detection and Verbatim vulnerability-detection template used in the production pipeline; patch verification promptsubstitutes the patched code block.

Instructions: Analyze this vulnerable code and return EXACTLY 5 CWE identifiers as a JSON list. Return ONLY a JSON array in this exact format: ["['CWE-XXX", "CWE-YYY", "CWE-ZZZ", "CWE-AAA", "CWE-BBB"]. NO explanations, NO descriptions, NO additional text. Use only identifiers (CWE-followed by numbers). **Order by likelihood** (most likely first).

Code Block Input:

```
{code}
```

Fig. 4. CWE ranking prompt.

using Cohen's thresholds [?] : $|r| < 0.1$ (negligible), $0.1 \leq |r| < 0.3$ (small), $0.3 \leq |r| < 0.5$ (medium), $|r| \geq 0.5$ (large (negligible $|r|$ <0.1, small, medium, large $|r|$ ≥0.5). This metric is crucial for our analysis because with highly imbalanced group sizes (Simple n=85 vs Complex n=18), even small actual differences can yield statistically significant p-values purely due to sample size . Rank-biserial correlation ensures we report both statistical significance *and* practical importance. matters under strong imbalance, where small differences can reach significance from sample size alone.

D. Prompt Templates

Our prompt templates for both SVD and CWE-label classification are designed with four key principles in mind: clear task specification, structured output formats, minimal introduction of bias, and scalability across vulnerability types. We focus on benchmarking the inherent LLM capabilities rather than optimizing prompting strategies, and on evaluating baseline performance to inform future research directions. Each As shown in Figure 3, each prompt clearly states the expected task and output format, including explicit three-way response instructions (`1`, `0`, or `not sure`; the uncertainty response is stored as `-1` for evaluation), uses standardized templates for fair comparison across models, maintains essential code context while avoiding information leakage, and reflects real-world security analysis scenarios.

Representative prompt formats are shown in Figures 3 (SVD) and 4 (CWE-label classification).

IV. Benchmarking Results

In this study, we leverage seven pre-trained LLMs to assess their capabilities in SVD and CWE-label classification tasks. Our selection comprises three code-specialized models (CodeLlama [?], StarCoder2 [?], and Qwen2.5-CoderQwen3-Coder [?]) explicitly trained on programming repositories, and four general-purpose models

TABLE IV
LLM SPECS WITH CONTEXT WINDOW SIZES (TOKENS).

Model	Type	# Params	Context Window
CodeLlama	Code-spec.	7B	16K
StarCoder2	Code-spec.	7B	16K
~~Qwen2.5-Coder~~ Qwen3-Coder	Code-spec.	30B	262K
Llama3.1	General	8B	131K
Mistral (v0.3)	General	7B	32K
DeepSeek-R1	General	7B	131K
GPT-4.1-mini	General	—	1M

(Llama3.1 [**?**], Mistral [**?**], and DeepSeek-R1 [**?**], GPT-4.1-mini [**?**]) designed for broader language understanding tasks. This balanced selection enables us to investigate whether domain-specific training provides advantages over general language comprehension for security-critical tasks. We selected these models based on five key criteria: (1) their state-of-the-art performance on code understanding and generation benchmarks, (2) architectural diversity spanning both standard fully-dense models (e.g., CodeLlama) and Mixture of Experts (MoE) models (e.g., DeepSeek-R1) to ensure comprehensive evaluation, (3) varying parameter sizes (7B-30B and unknown for GPT-4.1-mini) and context window capacities (16K-1M tokens) to investigate how model scale and memory affect SVD capabilities, (4) balanced representation of code-specialized versus general-purpose training to assess domain expertise effects, and (5) inclusion of both smaller open-source and larger proprietary models, allowing for a fair comparison across accessibility and capability dimensions. The specifications of the models we used are detailed in Table IV.

A. RQ1: Overall SVD performance

For this research question, we ~~divide our analysis into two parts. First, we analyze model performance in the~~ first separate valid binary verdicts from genuine abstentions and invalid outputs. We then analyze classification performance in two settings: non-targeted~~setting, where models are asked to classify code as vulnerable~~, where no specific CWE is provided, and targeted, where the prompt includes the CWE label. Because the validated response rates supplied for this audit are pooled over both settings and both datasets, Table V reports the overall temperature-0 response distribution rather than unsupported setting-specific estimates. Each available model contributes 1,668 responses (417 vulnerable/~~non-vulnerable with no information about the specific CWE . Then, we repeat the same analysis but for the targetedsetting, where models are given the CWE label in the prompt . Within each of these parts, we start by looking at how often each model actually provided an answer (regardless of its correctness) and how often it refused to answer (-1), and then look into the values for the metrics (accuracy, recall, and precision).~~ patched pairs evaluated in both settings).

1) ~~Non-targeted Classification~~: **Genuine abstention is rare, while most non-answers are formatting failures.** At temperature 0, Qwen3-Coder is the only model with observed genuine abstentions (0.06%). The other open-source models have no genuine abstentions. Invalid output is below 2% for CodeLlama, Mistral, and Qwen3-Coder, and absent

~~Abstention rates (%) for non-targeted and targeted SVD. **Models exhibit substantial differences in abstention behavior.** Table ?? shows abstention rates across both non-targeted and targeted settings. CodeLlama,~~

TABLE V
RESPONSE VALIDITY AT TEMPERATURE 0, POOLED OVER NON-TARGETED/TARGETED SETTINGS AND PBD/LFD. *Valid*: A RESOLVABLE 0/1 VERDICT; *Genuine abstention*: A CLEAR UNCERTAINTY VERDICT; *Invalid*: NON-COMPLIANT OR INDETERMINATE OUTPUT. GPT-4.1-MINI WAS NOT RUN AT THIS TEMPERATURE.

Model	
CodeLlama	
DeepSeek-R1	
GPT-4.1-mini	~~, and Mistral consistently provide predictions, with abstention rates below~~
Llama3.1	~~abstains in over half of all queries (52.6% –59.7%), refusing to commit to class~~
Mistral	
Qwen3-Coder-30B	
StarCoder2	

for DeepSeek-R1 and Llama3.1~~and Qwen2.5-Coder remain highly conservative across task types, revealing fundamental differences in model confidence calibration rather than task-specific uncertainty.~~ . StarCoder2 is the clear exception: 26.08% of its outputs are invalid, primarily because the base completion model continues code or otherwise fails to emit one of the requested verdicts. The broader decoding sweep in Table XIX confirms that genuine abstention and invalid output are distinct behaviors: GPT-4.1-mini averages 2.40% genuine abstention across the two temperature-0.1 runs, while StarCoder2's invalid rate reaches 26.08% at temperature 0.

1) Non-targeted Classification: **LLMs performed no better than random classifiers.** ~~In the non-targeted setting,~~ Table VI ~~(Non-Targeted columns) presents the actual performance metrics , considering only~~ presents the performance metrics for the cases where the models ~~did provide~~ provided a classification. ~~We can see that all~~ All models exhibit accuracies close to 50%, irrespective of architecture, size, or context window size. This ~~performance can be seen in both the~~ behaviour is observed on both PBD and LFD~~, indicating that potential training data leakage had no effect on performance and that none of the models have truly learned meaningful patterns to distinguish between vulnerable and non-vulnerable code in this setting~~; the historical split does not show a clear aggregate advantage over the post-cutoff split. Because the splits differ in composition and size, this comparison constrains but does not isolate the causal effect of training-data exposure.

Consider CVE-2024-26599 (CWE-119, PBD), a clear out-of-bounds array access in a PWM driver function (Figure 5). The vulnerable code ~~checks~~ validates that `args->args_count~~!= 1 && args->args_count != 2~~` ~~to validate input, then~~ is either 1 or 2, but subsequently accesses `args->args[2]` when `args_count == 2`. ~~This is an obvious bug: if an array has count=2~~Since arrays are zero-indexed, valid indices are only 0 and 1~~(zero-indexed), so index 2 is out of bounds. The fix simply changes~~ , making this an obvious out-of-bounds access. The patch simply replaces `args[2]` ~~to~~ with `args[1]`. A human reviewer can immediately identify this as vulnerable because it violates ~~basic array indexing rules. Yet all four major models failed: Codellama and Mistral incorrectly classified both~~ a basic array bounds constraint. Yet the models still struggled to

```
// File: drivers/pwm/core.c
of_pwm_single_xlate(struct pwm_chip *chip,
                    const struct of_phandle_args *args) {
   struct pwm_device *pwm;

   if (chip->of_pwm_n_cells < 1)
      return ERR_PTR(-EINVAL);

   // Validates that args_count is either 1 or 2
   if (args->args_count != 1 && args->args_count != 2)
      return ERR_PTR(-EINVAL);

   pwm = pwm_request_from_chip(chip, 0, NULL);
   if (IS_ERR(pwm))
      return pwm;

   pwm->args.period = args->args[0];
   pwm->args.polarity = PWM_POLARITY_NORMAL;

   // BUG: Accessing index 2 when count==2 (valid: 0,1)
- if (args->args_count == 2 && args->args[2] &
    ↪ PWM_POLARITY_INVERTED)
+ if (args->args_count == 2 && args->args[1] &
    ↪ PWM_POLARITY_INVERTED)
      pwm->args.polarity = PWM_POLARITY_INVERSED;

   return pwm;
}
```

Fig. 5. CVE-2024-26599: out-of-bounds access; patch shown.

distinguish the vulnerable and patched versions reliably. In the first run, GPT-4.1-mini correctly classified both the vulnerable and patched ~~versions~~ samples. CodeLlama, Mistral, and DeepSeek classified both as vulnerable, ~~Deepseek~~ while Llama3.1 classified both as safe~~, and Llama abstained on both. This demonstrates that the dataset contains clear, detectable vulnerabilities with sufficient context, but models simply lack the fundamental security reasoning to identify even straightforward bounds violations~~. StarCoder2 produced an invalid response for the vulnerable version and classified the patched version as safe. This example is consistent with the aggregate class biases in Table VI: even when the local defect is explicit, several models preserve the same verdict across the vulnerable–patched pair rather than responding to the changed array index.

These results stand in sharp contrast to supervised detection tools evaluated under different assumptions: SecureFalcon [**?**] reports 94% binary accuracy on C/C++ code after training on FormAI/FalconVulnDB, and Alam et al. report 99% Solidity vulnerability-detection accuracy after fine-tuning GPT-3.5 Turbo and GPT-4o Mini [**?**]. These figures are not directly comparable because the studies use task-specific training datasets and different software domains, whereas our evaluation measures zero-shot, off-the-shelf performance and includes a post-cutoff split.

Models ~~tend to have an extreme bias towards one of the classes~~exhibit strong and consistent class biases rather than meaningful vulnerability detection behaviour. Examining ~~metrics such as~~ Recall, Precision, and PPR reveals ~~consistent~~ clear patterns across both ~~the PBD and LFD . CodeLlama and Mistralstand~~ out in both datasets with ~~datasets.~~ In the non-targeted setting, CodeLlama, DeepSeek, Mistral, and Qwen3-Coder all achieve very high recall values (~~around 75–88%for PBD and 70–81% for LFD) , indicating that they correctly flag most vulnerable samples~~ roughly 70–85%) together with high PPR values, indicating a strong tendency to classify most samples as vulnerable. However, their ~~corresponding precision and PPR values (around~~ precision remains close to 50%~~precision and 0.8 PPR) indicate~~ , showing that these high recall scores are ~~driven by a strong bias toward positive predictions: the models tend to classify most samples as vulnerable, regardless of the true label.~~

largely driven by overpredicting the positive class rather than correctly identifying vulnerabilities. In contrast, ~~models such as~~ GPT-4.1-mini ~~,~~ and Llama3.1 ~~, Qwen2.5-Coder, and DeepSeek~~ exhibit the opposite behaviour~~across both datasets. Their~~ , with low recall and ~~PPR values indicate~~ low PPR values across both datasets, indicating a strong bias toward ~~the negative class (~~predicting samples as non-vulnerable~~code), with most predictions defaulting to . Yet despite~~ . Despite this conservative tendency, ~~their precision remains low, so even when these models predict a vulnerability, they are often incorrect.~~

~~Overall, these patterns are remarkably consistent across the PBD and LFD, reinforcing the conclusion that the observed biases are intrinsic to the models rather than a result of training data leakage or dataset-specific artifacts~~precision also remains close to random, meaning that even its positive predictions are unreliable. StarCoder2 appears comparatively more balanced, with recall and PPR values closer to 50%, but its overall accuracy still remains at chance level.

2) Targeted Classification: ~~Moving to the targeted classification~~ **Providing the CWE label shifts model biases but does not improve classification performance.** In the targeted setting, Table **??** ~~presents the response distributions across models.~~

~~Providing the CWE label influenced models' abstention behavior in different ways.~~ ~~For DeepSeek and StarCoder2, including the CWE label led to fewer abstentions. Specifically, DeepSeek's proportion of predicted responses increased from 71.1% to 93.65% ,~~ VI (Targeted columns) summarizes the performance metrics. Accuracy remained near 50% for all models, so providing the CWE label did not produce a clear aggregate improvement. However, examining recall and PPR reveals notable bias shifts. CodeLlama and DeepSeek saw their positive class bias further amplified, with recall rising to around 88% and 87% on PBD respectively, while precision remained near 50%. Mistral exhibited the most dramatic reversal, flipping from a strong positive bias (recall: 85.2%) to a strong negative one (recall: 19.6%). Llama3.1's existing negative bias deepened further, while Qwen3-Coder moved toward balance, with recall dropping from 84.6% to 61.6% on PBD. StarCoder2 ~~'s increased from 81.77%to 86.21%~~and GPT-4.1-mini showed a modest increase in positive bias. Thus, CWE labels shift models in distinct and sometimes opposing directions without a consistent accuracy gain. ~~In contrast, for Qwen2.5-Coder, the proportion of predicted responses declined from 62.47% to 53.48% . For the remaining four models, assigning the CWE labels did not lead to substantial changes in abstention frequency. Overall, this suggests that the inclusion of CWE labels had model-dependent effects on prediction behavior. Some models became more decisive, while others grew more cautious, highlighting variability in how each model interprets and leverages the task information.~~

TABLE VI
PERFORMANCE METRICS FOR SVD IN NON-TARGETED AND TARGETED SETTINGS.

Model	Non-Targeted											
	PBD (%)				LFD (%)				PBD (%)			
	Acc.	Rec.	Prec.	PPR	Acc.	Rec.	Prec.	PPR	Acc.	Rec.	Prec.	
CodeLlama	~~49.0~~ 49.9	~~75.7~~ 73.3	~~49.4~~ 49.9	~~76.8~~ 73.5	~~51.0~~ 50.5	~~70.9~~ 69.9	~~50.7~~ 50.3	~~69.9~~ 69.4	~~52.2~~ 51.7	~~92.0~~ 88.2	~~51.2~~ 51.0	~~8~~
DeepSeek	~~51.3~~ 49.6	~~16.0~~ 81.3	~~49.2~~ 50.1	~~15.7~~ 80.1	~~46.9~~ 50.0	~~11.5~~ 77.7	~~52.9~~ 50.0	~~11.7~~ 77.7	~~46.5~~ 49.6	~~44.5~~ 87.3	~~44.7~~ 50.2	~~4~~
GPT-4.1-mini	~~50.7~~ 50.8	~~5.8~~ 5.2	~~60.0~~ 59.9	~~4.9~~ 4.4	~~51.8~~ 51.5	~~4.2~~ 5.5	~~66.7~~ 70.0	~~3.1~~ 4.0	~~49.5~~ 49.6	~~7.8~~ 7.4	~~49.0~~ 47.3	
Llama3.1	~~45.0~~ 50.1	~~3.1~~ 31.6	~~33.3~~ 50.1	~~5.0~~ 31.5	~~40.9~~ 51.0	~~0.0~~ 29.4	~~0.0~~ 51.6	~~0.0~~ 28.5	~~46.7~~ 49.4	~~0.0~~ 27.3	~~0.0~~ 48.9	
Mistral	49.4	~~88.2~~ 85.2	~~49.6~~ 49.7	~~88.9~~ 85.8	~~51.0~~ 50.5	~~80.6~~ 79.9	~~50.6~~ 50.3	~~79.6~~ 79.4	~~47.0~~ 47.7	~~20.1~~ 19.6	~~43.4~~ 45.0	~~2~~
~~Qwen2.5-Coder~~ Qwen3-Coder	~~53.8~~ 50.3	~~4.8~~ 84.6	~~62.5~~ 50.3	~~3.6~~ 84.4	~~42.9~~ 49.0	~~2.6~~ 79.6	~~33.3~~ 49.4	~~4.3~~ 80.6	~~52.9~~ 50.0	~~0.0~~ 61.6	~~0.0~~ 50.2	
StarCoder2	~~48.2~~ 49.0	~~18.9~~ 52.2	~~44.4~~ 47.0	~~21.0~~ 52.7	~~54.7~~ 49.9	~~15.8~~ 48.8	~~47.4~~ 49.5	~~14.8~~ 49.3	~~45.4~~ 49.2	~~34.3~~ 61.4	~~43.5~~ 48.4	~~3~~

B. *RQ2: Consistency across CWEs pillars*

~~**Similar effects can be seen in the**~~ **LLMs show weak, near-chance performance** ~~**metrics**~~**across CWE pillars, reinforcing that they are not yet capable of reliable SVD classification.** In the ~~targeted setting, Table VI (Targeted columns) summarizes the performance metrics for SVD. Overall, the accuracies remained around 50%, indicating that even when the corresponding CWE label was provided, the models did not perform better than~~ non-targeted setting, performance varies slightly by pillar but remains close to random guessing. ~~However, when examining the other metrics, some interesting trends emerge. For DeepSeek, recall increased from 16.0% to 44.5% , while the PPR rose from 15.7%to 48.2%. This suggests that, although overall performance did not improve, providing the label helped DeepSeek overcome its initial bias toward the negative class. A similar, albeit smaller, effect can be observed for StarCoder2, where recall increased from 18.9% to 34.3% and PPR from 21.0% to 39.1%. For Mistral, the opposite trend occurred: recall dropped from 88.2%to 20.1% and PPR from 88.9% to 23.1%. In this case, performance again did not meaningfully improve, but the bias shifted entirely from favoring the positive class to favoring the negative one. For the remaining models, no significant changes were observed across any of the metrics.~~ For PBD, accuracies range from 48.3% (CWE-OTHER) to 52.7% (CWE-693), with most pillars clustered near 50%. For LFD, results are only available for a subset of pillars (CWE-664/682/691/707/710) and accuracies range from 49.3% to 54.9%. Beyond accuracy, the other metrics show the same overall weakness: in PBD, recall spans 55.2%–60.5% and precision ranges from 48.7% to 51.8%.

~~These findings once again demonstrate that the inclusion of CWE labels affects models in distinct and sometimes opposing ways, suggesting differences in how each model integrates structured contextual information during classification~~The same trends can be seen for LFD and for the targeted setting, with models always remaining near chance across pillars, as summarized in Table VII.

~~**LLMs show weak, near-chance performance across CWE pillars, reinforcing that they are not yet capable of reliable SVD classification.** In the non-targeted setting, performance varies slightly by pillar but remains close to random guessing. For PBD, accuracies range from 43.1% (CWE-707) to 55.2% (CWE-710), with most pillars clustered near 50%. For LFD, results are only available for a subset of pillars (CWE-664/682/691/707/710) and accuracies range from 40.0% to 56.1%. Beyond accuracy, the other metrics show the same overall weakness: in PBD, recall spans 28.6%–51.5% and precision stays roughly in the mid-40s to high-50s, while in LFD recall can be as low as 0.0% (CWE-691), indicating highly unstable behavior in low-support categories.~~

~~The targeted setting does not consistently improve performance. While some pillars show higher accuracy (e.g., CWE-693 in PBD rises to 58.3%, and CWE-664 in LFD rises to 55.0%), others degrade (e.g., CWE-682 in PBD drops to 44.3%, and CWE-707 in LFD drops to 51.2%). Overall, models remain near chance across pillars in both settings, as summarized in Table VII.~~

C. *RQ3: ~~Effect of Code Abstraction Level~~Consistency across granularity levels*

Real-world vulnerabilities rarely exist in isolation. A single vulnerable function often depends on context from other functions, global variables, macros, or configuration settings spread across multiple files. To capture this reality, we evaluate models at three code abstraction levels: *Level 1* presents only the modified function in isolation; *Level 2* includes all functions modified in the security patch within a single file; and *Level 3* provides the complete context across multiple files.

Our comprehensive analysis across three abstraction levels reveals surprising patterns that challenge conventional assumptions about SVD complexity. We evaluated both PBD and LFD to ensure robust findings.

Abstraction Does Not Always Hinder Performance, But Individual Models Diverge~~Dramatically~~. ~~Table VIII reveals a paradox: while mean accuracy remains stable~~ Mean accuracy remains similar across abstraction levels (PBD: ~~48.9~~49.0–49.~~6~~9%; LFD: ~~47.6–51.3~~47.2–50.9%), ~~statistical tests confirm no significant difference between levels~~and the omnibus tests do not reject equal group distributions. For PBD, one-way ANOVA yields F=~~0.050~~ 0.432 (p=~~0.9513), with the F-statistic well below 1.0, indicating that variance between abstraction levels is even smaller than within-level variance~~0.6558). For LFD, where normality assumptions fail (Shapiro-Wilk p=0.008 for L3), ~~we use the non-parametric~~ the Kruskal-Wallis test (yields H=~~0.427, 2.995~~ (p=~~0.8078). Combined with high p-values (both $\gg$0.05), this confirms that observed accuracy differences are indistinguishable from random noise. However,~~

TABLE VII
PER-CWE BREAKDOWN OF THE PERFORMANCE METRICS FOR SVD.

CWE Pillar	Non-Targeted								Targeted			
	PBD (%)				LFD (%)				PBD (%)			
	Acc.	Rec.	Prec.	PPR	Acc.	Rec.	Prec.	PPR	Acc.	Rec.	Prec.	PPR
CWE-284	~~50.0~~ 48.8	~~38.7~~ 59.4	~~52.2~~ 48.7	~~38.3~~ 60.6	–	–	–	–	~~45.0~~ 47.0	~~32.3~~ 46.4	~~45.5~~ 47.0	~~36.7~~ 49.4	
CWE-664	~~47.9~~ 50.7	~~48.0~~ 60.0	~~50.0~~ 50.5	~~50.0~~ 59.1	~~48.9~~ 49.7	~~30.2~~ 54.0	~~45.2~~ 49.9	~~32.1~~ 54.3	~~51.3~~ 49.7	~~35.1~~ 50.4	~~50.0~~ 49.8	~~34.2~~ 50.6	~~55.~~
CWE-682	~~54.1~~ 49.0	~~51.5~~ 59.6	~~58.6~~ 49.2	~~47.5~~ 60.6	~~40.0~~ 51.4	~~33.3~~ 63.0	~~50.0~~ 51.3	~~40.0~~ 61.6	~~44.3~~ 51.1	~~27.3~~ 49.0	~~47.4~~ 51.3	~~31.1~~ 47.9	~~40.~~
CWE-691	~~46.2~~ 48.7	~~28.6~~ 55.2	~~50.0~~ 48.8	~~30.8~~ 56.5	~~44.4~~ 49.3	~~0.0~~ 57.8	~~0.0~~ 49.3	~~0.0~~ 58.5	~~53.8~~ 50.3	~~35.7~~ 49.4	~~62.5~~ 50.2	~~30.8~~ 49.1	~~44.~~
CWE-693	~~54.2~~ 52.7	~~47.6~~ 60.5	~~47.6~~ 51.8	~~43.8~~ 57.7	–	–	–	–	~~58.3~~ 48.9	~~47.6~~ 48.6	~~52.6~~ 48.8	~~39.6~~ 49.7	
CWE-703	~~49.1~~ 50.7	~~46.4~~ 57.0	~~48.1~~ 50.2	~~47.4~~ 56.2	–	–	–	–	~~47.4~~ 48.6	~~39.3~~ 51.3	~~45.8~~ 48.6	~~42.1~~ 52.8	
CWE-707	~~43.1~~ 50.1	~~42.3~~ 60.5	~~44.0~~ 50.6	~~49.0~~ 60.6	~~56.1~~ 54.9	~~40.9~~ 60.5	~~64.3~~ 53.9	~~34.1~~ 55.5	~~43.1~~ 50.2	~~38.5~~ 57.6	~~43.5~~ 50.0	~~45.1~~ 57.3	~~51.~~
CWE-710	~~55.2~~ 48.6	~~38.5~~ 57.4	~~50.0~~ 48.8	~~34.5~~ 58.5	~~51.7~~ 50.7	~~40.0~~ 56.4	~~54.5~~ 50.3	~~37.9~~ 55.7	~~56.9~~ 47.7	~~42.3~~ 34.4	~~52.4~~ 47.4	~~36.2~~ 36.4	~~48.~~
CWE-OTHER	~~49.0~~ 48.3	~~38.2~~ 55.3	~~49.1~~ 48.9	~~39.5~~ 56.6	–	–	–	–	~~49.1~~ 50.2	~~38.8~~ 55.3	~~49.0~~ 50.5	~~40.2~~ 54.8	

~~the standard deviation tells a completely different story. In LFD, model agreement collapses as abstraction increases: Std grows from 1.96~~0.2237). These non-significant results do not prove equivalence; they indicate that the study lacks evidence of a dataset-wide abstraction effect. Descriptively, LFD's between-model standard deviation grows from 1.10% at Level 1 to ~~7.93~~6.80% at Level ~~3, a 4× explosion in variance. This divergence exposes that models respond in fundamentally opposite ways to increased context.~~ 3. Table IX ~~quantifies this split: Codellama *improves* by 2.6% in PBD with added abstraction, yet *catastrophically degrades* by 22.0~~shows the corresponding heterogeneity: CodeLlama changes little in PBD but declines by 13.4% in LFD~~. Meanwhile, Deepseek improves in *both* datasets (−1.7% PBD, −6.9% LFD), and Starcoder consistently struggles (5.1% PBD, 8.9% LFDdegradation). The stable aggregate statistics mask wildly different model behaviors: some architectures leverage additional context effectively while others drown in it, demonstrating that abstraction robustness is~~ DeepSeek declines by 3.2% in PBD and 4.7% in LFD, and StarCoder2 improves by 8.7% in LFD. Thus abstraction sensitivity is strongly model-specific ~~rather than a universal property of LLMs~~in these samples.

~~**Model Specialization Drives Robustness.Table IX reveals that model specialization significantly impacts abstraction sensitivity. In PBD, three models *improve* with increased abstraction: Codellama (−2.6%), Deepseek (−1.7%)**~~**Group Averages Differ Modestly**~~**, and Qwen3 (−0.8%), while Starcoder shows the highest degradation (5.1%)**~~**but Within-Group Variation Dominates.General-purpose models demonstrate superior robustness with 0.4% average degradation versus 8.7% for code-specialized models in PBD. This advantage becomes even more pronounced in LFD:~~** Code-specialized models average 0.4% degradation in PBD and 2.6% in LFD (1.5% across splits), compared with 1.1% and 4.5% for general-purpose ~~models maintain near-stable performance with only 0.3% degradation, while code-specialized models degrade by 8.8%, resulting in an 8.5%performance gap.~~ models (2.8% across splits). With only three and four models per group, these descriptive averages do not establish a specialization effect. Within-group variation is larger: StarCoder2 improves by 3.8% on average as abstraction increases, while CodeLlama degrades by 6.8%; GPT-4.1-mini averages 1.0% degradation, whereas Llama3.1 averages 4.8%. The evidence therefore supports model-specific sensitivity more strongly than a general code-specialization advantage.

TABLE VIII
OVERALL PERFORMANCE BY ABSTRACTION LEVEL: MEAN ACCURACY AND STANDARD DEVIATION ACROSS MODELS.

Abstraction Level	PBD		LFD	
	Mean (%)	Std. (%)	Mean (%)	Std. (%)
Level 1 (Single Function)	49.8	0.94	50.9	1.10
Level 2 (Multiple Functions)	49.9	2.77	50.6	3.65
Level 3 (Multiple Files)	49.0	1.98	47.2	6.80

Statistical tests: PBD: One-way ANOVA F=0.432, p=0.6558;
LFD: Kruskal-Wallis H=2.995, p=0.2237

TABLE IX
MODEL ROBUSTNESS BY ABSTRACTION SENSITIVITY. DEGRADATION = L1 − L3 ACCURACY; NEGATIVE INDICATES IMPROVEMENT. RANKED BY AVERAGE DEGRADATION (WORST → BEST).

Rank (worst → best ↓)	Model	Type	Context Window	Degradation (%) PBD	LFD	Avg
1	Codellama	Code-Spec.	16K	0.2	13.4	6.8
2	Llama	General	128K	−2.1	11.6	4.8
3	Deepseek	General	64K	3.2	4.7	4.0
4	Qwen3	Code-Spec.	32K	0.1	3.2	1.6
5	Mistral	General	32K	3.2	−0.3	1.4
6	GPT-4.1-mini	General	1M	0.2	1.9	1.0
7	Starcoder	Code-Spec.	16K	1.0	−8.7	−3.8
Code-Specialized Avg.				0.4	2.6	1.5
General-Purpose Avg.				1.1	4.5	2.8

Context Window Size Does Not Guarantee Robustness. ~~Table IX reveals that~~ GPT-4.1-mini, despite having the largest context window (1M tokens), ~~ranks only 4th with 1.0% degradation , being outperformed by models with far smaller contexts: Deepseek (−1.8% , 64K), Qwen3 (−2.1%, 32K), and Mistral (0.0%, 32K)~~shows more degradation than StarCoder2 (1.0% vs. −3.8%), which has a 16K-token window. Pearson correlation ~~analysis confirms this disconnect: context window size shows no statistically significant relationship with abstraction robustness~~ is not statistically significant in either split (PBD: r=−0.365, p=0.635; LFD: r=−0.592, p=0.408). ~~Architectural design matters more than raw context capacity~~With seven models, this analysis only shows that context-window size alone does not explain the observed robustness pattern.

Task-Specific Patterns ~~Emerge~~Are Descriptive, Not Statistically Conclusive. Mann-Whitney U tests in Ta-

TABLE X
TASK-SPECIFIC ACCURACY ACROSS ABSTRACTION LEVELS AND L1 → L3 CHANGE. EXCLUDED ABSTENTION CASES.

Task	PBD Accuracy (%)				LFD Accuracy (%)			
	L1	L2	L3	Δ (L1→L3)	L1	L2	L3	Δ (L1→L3)
Non-Targeted	50.0	50.4	48.8	1.2	50.4	49.9	45.2	5.2
Targeted	49.6	49.4	49.2	0.4	51.4	51.4	49.2	2.2

Statistical tests (LFD, Simple vs Complex):
Non-Targeted: Mann-Whitney U=69, p=0.1442, r=0.326 (medium effect)
Targeted: Mann-Whitney U=56, p=0.6272, r=0.114 (small effect)

ble X demonstrates distinct sensitivities between targeted and non-targeted tasks after correcting for -1 (no answer) handling. In PBD, both task types show similar modest degradation: Non-Targeted degrades from 50.7% (compare Simple (L1) to 46.9% (with Complex (L2 and L3, 3.8% loss) while Targeted degrades from 50.1% to 45.9% (4.1% loss). LFD reveals comparable patterns: Non-Targeted degrades by 3.6% (50.8%→47.2%) and Targeted by 5.2% (52.4%→47.2%). Mann-Whitney U tests on the LFD Simple vs Complex comparison show that both task types exhibit performance decline with increased abstraction, with Non-Targeted showing a medium) LFD samples. Non-targeted tasks show a medium estimated effect (U=6669, p=0.08280.1442, r=0.363)and Targeted showing 0.326), while targeted tasks show a small effect (U=6056, p=0.24480.6272, r=0.257). Though neither reaches statistical significance (p<0.05) , the consistent degradationpatterns across both task types suggest that CVE/CWE context provides limited protection against abstraction complexity in this corrected analysis . 0.114); neither comparison is statistically significant. Descriptively, targeted tasks exhibit 0.4% (PBD) and 2.2% (LFD) L1-to-L3 degradation, compared with 1.2% and 5.2% for non-targeted tasks. This pattern motivates further study of whether CWE context helps focus analysis in larger code scopes, but the present data do not establish that mechanism.

The imbalanced distribution of abstraction levels in LFD (L1: 82.5%, L2: 14.6%, L3: 2.9%) reflects natural real-world patterns but limits statistical power for fine-grained analysis. Our binary comparison (Simple vs Complex) comparison in Table X addresses this limitation by using appropriate non-parametric methods and reporting effect sizes, but future work should include the targeted collection of multi-component, multi-file vulnerabilities to enable more robust three-way comparisons. Nevertheless, our finding that the abstraction level does *not* consistently affect average performance (as shown in Tables IX and X), combined with the dramatic model-specific divergence in robustness, provides valuable insights for deployment strategies for vulnerability detection.

Mann-Whitney U tests in Table X compare Simple (L1) vs Complex (L2 and L3) abstraction levels for the LFD dataset. Non-Targeted tasks show a medium effect size (U=66, p=0.0828, r=0.363), approaching statistical significance, while Targeted tasks show a small effect (U=60, p=0.2448, r=0.257). Both task types consistently show performance decline with increased abstraction: Targeted tasks exhibit 5.2% degradation (L1 to L3) compared to Non-Targeted tasks' 3.6% decline. Though neither reaches statistical significance (p<0.05), the consistent degradation across both task types indicates that CVE/CWE context provides limited robustness benefits against abstraction complexity. Notably, Targeted detection shows greater degradation despite being inherently more difficult—it requires models to identify both whether code is vulnerable *and* classify the specific vulnerability type, creating a more constrained search space. When CVE/CWE context is added, models must maintain both the specific vulnerability signature and broader code dependencies across functions and files, compounding the cognitive load. This finding suggests that semantic anchors from vulnerability-specific context alone are insufficient to compensate for the challenges of understanding vulnerabilities across multiple components, particularly in the leakage-free setting where models cannot rely on memorized CVE-code patterns.

Code-Specialized Models Show Greater Abstraction Sensitivity. Tables IX and X reveal significant differences between model types. In Table IX, code-specialized models show substantially worse robustness: they degrade by 8.7% in PBD and 13.3% in LFD (average 11.0%). In contrast, general-purpose models show remarkable stability: only 0.4% degradation in PBD and 0.0% in LFD (average 0.2%). This 10.8% gap demonstrates that specialization for code tasks does not inherently improve robustness to abstraction complexity. Individual models show dramatic divergence: Deepseek (general-purpose) actually *improves* by −1.8% on average as abstraction increases, while Codellama (code-specialized) degrades by 22.0% in both datasets. The consistency of Codellama's poor performance across both PBD and LFD (22.0% in each) suggests an architectural limitation rather than mere training data memorization. These findings indicate that general-purpose models' broader training may provide better transfer learning for complex code understanding tasks.

Note that, while LFD provides a valuable signal for evaluating model behavior on post-cutoff vulnerabilities, its size is inherently limited (103 samples across 19 CWEs, with very sparse coverage at higher abstraction levels). Consequently, results obtained on LFD should be interpreted as *indicative rather than definitive* of generalization capability. Observed performance differences and degradation trends highlight meaningful failure modes, but small sample sizes may amplify variance and reduce statistical power, particularly for fine-grained analyses.

D. RQ4: CWE Ranking Performance

Identifying the specific vulnerability type is critical for security practitioners. When a model detects vulnerable code, , as knowing the exact CWE category helps developers understand the root cause and apply appropriate fixes. CWE ranking evaluates whether models can not only detect vulnerabilities but also correctly classify them within the CWE taxonomy. This task is challenging because the CWE system is hierarchical, with over 900 weakness types organized into four levels: Pillars(highest-level categories like "Resource Control"), Classes, Base weaknesses, and Variants(specific implementations). We focus on root pillars , the highest level of this hierarchy, because they represent fundamental security concepts that guide remediation strategies. Even

TABLE XI
CWE RANKING ACROSS PBD AND LFD, AVERAGED ACROSS THREE RUNS. COLUMNS REPORT TOP-k ACCURACY, MRR, AND HPS.

Model	PBD: CWE Top-k (%)					LFD: CWE Top-k (%)					MRR (%)	
	T_1	T_2	T_3	T_4	T_5	T_1	T_2	T_3	T_4	T_5	PBD	LFD
CodeLlama	0.5	0.8	1.0	1.0	1.3	1.6	2.3	2.3	2.6	6.5	0.9	3.0
DeepSeek	0.3	0.5	0.7	0.8	1.2	0.0	0.3	0.3	0.3	0.3	0.6	0.2
~~Llama3~~GPT-4.1-mini	**3.3**	**5.8**	**7.6**	**9.5**	**10.7**	**15.6**	**26.2**	**41.5**	**51.7**	**52.7**	**5.8**	**28.8**
~~Mistral~~ Llama3.1	1.2	2.1	4.4	6.5	7.3	1.3	5.5	9.1	11.7	14.9	3.1	5.9
~~Qwen2.5-Coder~~ Mistral	1.1	2.4	3.2	5.3	6.9	5.2	7.2	8.8	9.8	10.4	2.9	7.1
~~StarCoder2~~ Qwen3-Coder-30B	2.2	4.8	6.7	7.8	10.0	6.2	14.6	18.2	23.1	31.2	4.9	14.4
~~GPT-4.1-mini~~ StarCoder2	1.3	3.1	4.4	6.5	6.7	0.0	0.6	0.6	3.6	5.9	3.2	1.6
Average	~~1.3~~ 1.4	~~2.7~~ 2.8	~~4.2~~ 4.0	5.3	~~6.7~~ 6.3	~~2.7~~ 4.3	~~5.3~~ 8.1	~~5.3~~ 11.5	~~11.6~~ 14.7	~~11.8~~ 17.4	~~3.2~~ 3.0	~~5.6~~ 8.7

~~partial understanding (identifying the correct pillar when the exact variant is wrong) provides value for security triage.~~ . We retain the sampled fine-grained CWE identifiers for ranking and use root pillars only to organize coverage and hierarchy-aware relations.

We evaluate models ~~using~~ on the CWE ranking task~~(SVD2)~~, where models rank their top-5 most likely CWE predictions for ~~vulnerable code. We measure performance using three metrics:~~ a given vulnerable code snippet. Performance is measured using Top-k accuracy~~(whether the correct CWE appears in the top-k predictions), (how highly models rank the correct answer)~~, MRR, and HPS~~(which rewards semantically similar~~ , which rewards taxonomically related predictions based on ~~CWE hierarchy)~~the CWE hierarchy. Table XI summarizes ~~performance across both PBD and LFD~~results across both datasets. We opted for a ranking formulation instead of binary per-CWE classification because the dataset spans 74 distinct CWE identifiers, many with fewer than 10 examples, making individual binary classifiers statistically unreliable. The Top-k accuracy and HPS metrics together capture exact identification and partial taxonomic credit.

~~Models show Semantic Understanding of Vulnerability Families~~ HPS Reveals Limited Taxonomic Proximity Despite Low Exact Accuracy. Table XI reveals that precise CWE identification is far harder than binary vulnerability detection. Even the best model, GPT-4.1-mini, achieves only ~~2.92~~3.3% Top-1 accuracy in PBD and ~~14.71~~15.6% in LFD. ~~Most models perform dismally: CodeLlama and DeepSeek manage~~ DeepSeek manages less than 1% Top-1 accuracy in both datasets ~~. However,~~ (0.3% PBD, 0.0% LFD), while CodeLlama achieves 0.5% in PBD and 1.6% in LFD. Top-5 performance ~~tells a more nuanced story~~is higher: GPT-4.1-mini reaches ~~50.98~~52.7% in LFD, ~~suggesting models can narrow down possibilities even when they miss the exact CWE. Interestingly,~~ showing that the exact label is sometimes present among its candidates even when not ranked first. CWE ranking exhibits mixed patterns: GPT-4.1-mini and ~~Qwen2.5-Coder actually~~ Qwen3-Coder improve in LFD (approximately 5× and ~~4~~3× better Top-1 accuracy, respectively), while ~~code-specialized models like DeepSeek and~~ StarCoder2 ~~collapse to near-zero performance. The gap between HPS and~~ and DeepSeek reach 0.0% Top-1 ~~accuracy (17.32~~in LFD. The HPS–Top-1 gap (19.3% vs.~~2.92~~ 3.3% for GPT-4.1-mini in PBD~~, see~~ ; Table XI) ~~indicates that models frequently predict CWEs within the correct taxonomy branch even when the exact classification is wrong. While only 13.8% of incorrect predictions are hierarchically close (distance ≤3, Table XV)~~records some taxonomically related candidates, but only 13.9% (289/2,~~this *taxonomy-aware prediction* has practical value for security triage, as it narrows the vulnerability family (e.g., correctly identifying buffer-related issues vs access control issues)~~077) of incorrect Top-1 predictions are within hierarchy distance 3 (Table XVIII). Such near misses may help route manual review, but the low rate does not establish broad semantic understanding. However, the large HPS-to-Top-1 ratios (5.9~~× to 15.7×,~~ × to 17.2× in PBD; Table XVII) and systematic CWE-119 bias (~~45.1~~44.2% appearance rate, with Llama reaching 98.6%, Table XVI) suggest this may reflect taxonomy pattern-matching rather than genuine semantic understanding.

Frequent Vulnerability Types ~~Inflate~~ Are Associated with Higher Aggregate Performance~~Metrics~~. Table XII ~~reveals that models perform significantly better on common CWE types than rare ones~~shows that aggregate scores are generally higher when frequent CWE types receive more weight. As defined in §III-C, micro accuracy weights all samples equally (~~favoring~~ thereby emphasizing frequent types), while macro accuracy averages performance across ~~all~~ CWE classes equally. ~~The gap between these metrics exposes frequency bias.~~ On average, the ~~gap is small in PBD (1.1 points) but jumps dramatically in LFD(6.1 points) (a 5.5× increase)~~micro–macro gap is 0.9 points in PBD and 6.6 points in LFD. This pattern ~~shows data leakage hides the bias~~is consistent with sensitivity to class frequency, but the difference between splits may also reflect their different class composition and the smaller LFD sample. Individual models show ~~even~~ stronger effects: GPT-4.1-mini achieves ~~51.0~~52.7% micro but ~~only 24.4~~25.3% macro in LFD (~~26.6~~ 27.4-point gap), ~~meaning it performs well on common vulnerabilities but fails on rare types. Qwen2.5-Coder shows similar behavior with a 9.3 gap (30.6~~while Qwen3-Coder shows an 11.5-point gap (31.2% vs.~~21.3~~ 19.7%). ~~Interestingly, two models show negative gaps in LFD (Llama3.1: -0.1, Mistral: -1.3), performing slightly better on rare types (possibly because they rely less on memorization). The dramatic increase in the gap from PBD to LFD confirms that models learn frequency-based patterns rather than generalizable security knowledge.~~ Mistral instead has a small negative LFD gap (−1.1), indicating slightly higher average performance across classes than across samples. These descriptive differences do

TABLE XII
MICRO/~~MACRO~~ MACRO TOP-5 EXACT-INCLUSION ACCURACY ~~ANALYSIS REVEALING MODEL PERFORMANCE DISPARITIES~~ ACROSS CWE TYPES. GAPS ARE COMPUTED FROM UNROUNDED THREE-RUN MEANS AND MAY DIFFER BY 0.1 POINT FROM SUBTRACTING THE DISPLAYED VALUES.

Model	PBD			LFD		
	Micro (%)	Macro (%)	Gap (pp)	Micro (%)	Macro (%)	Gap (pp)
CodeLlama	2.1	2.2	−0.1	7.5	3.3	4.2
DeepSeek	1.2	1.0	0.1	0.7	0.5	0.1
Llama3.1	7.3	5.9	1.4	14.9	14.7	0.2
Mistral	6.9	5.5	1.5	10.4	11.5	−1.1
Qwen3-Coder	10.0	7.7	2.4	31.2	19.7	11.5
StarCoder2	6.8	5.5	1.3	6.5	2.6	4.0
GPT-4.1-mini	10.7	10.7	0.0	52.7	25.3	27.5
Average	6.4	5.5	0.9	17.7	11.1	6.6

not identify a particular internal model mechanism. Per-CWE estimates, especially for classes with fewer than approximately 20 samples, should therefore be treated as indicative rather than statistically definitive.

~~Models Excel at Memory Safety but Fail at Configuration Vulnerabilities~~Performance Concentrates on Input-Validation and Memory-Related CWEs. Table~~XIII shows models perform best on memory-safety vulnerabilities but have clear blind spots. In PBD , four out of five top performers are buffer-related (~~ XIII shows the highest PBD averages for CWE-119 ~~, CWE-122, CWE-120, plus CWE-20), revealing training data bias toward common memory vulnerabilities in C/C++ codebases. CWE-119 ~~(Buffer Errors, 44.8%), CWE-20 (Improper Input Validation, 34.2%)~~achieves 57.1%accuracy~~, CWE-122 (Heap Overflow~~) at 40.8%~~, and CWE-120 (Buffer Copy) at 23.8%. These vulnerabilities have clear syntactic patterns that models can learn. However, five CWEs remain completely ~~34.0%), CWE-129 (Improper Array Index Validation, 21.9%), and CWE-190 (Integer Overflow, 21.4%). Five reported CWEs remain undetected at 0%: ~~CWE-327 (Broken Crypto), CWE-16 (Configuration), CWE-732 (Incorrect Permissions), CWE-1284 (Improper Validation~~CWE-191 (Integer Underflow), CWE-276 (Incorrect Default Permissions), CWE-665 (Improper Initialization), CWE-755 (Improper Handling of Exceptional Conditions), and ~~CWE-326 (Inadequate Encryption). These vulnerabilities often require inter-procedural or system-level analysis that a single-function context cannot provide. LFD shows similar patterns but with lower overall performance: the best CWE-763 (Release of Invalid Pointer). LFD shows a related concentration, with CWE-20 (Input Validation) score is only 35.7%, compared to PBD's 57.1%. Both datasets show that models struggle with semantic vulnerabilities that require domain knowledge of cryptography, permissions, and configuration, while succeeding at syntactic patternssuch as buffer operations~~reaching 34.5%. These descriptive results are compatible with several explanations, including class frequency, training exposure, and the visibility of local code patterns; the benchmark does not disentangle those causes, especially for rare CWEs.

CWE-119 Acts as a ~~Semantic Magnet~~ Frequent Destination for Diverse ~~Vulnerability Types~~Errors. Table~~XIV reveals models consistently misattribute diverse vulnerability types to~~ XIV shows that all top-eight PBD confusion pairs target CWE-119 (Buffer Errors)~~. In PBD, all top-8 confusion pairs target CWE-119~~, spanning heap overflow (CWE-122, ~~19~~ 17.7 instances), ~~access control~~ incorrect permissions (CWE-276, ~~15 instances~~15.3), buffer copy (CWE-120, ~~14 instances), and authorization issues (CWE-862, 13 instances). The diversity of source CWEs, including resource management issues such as CWE-772~~13.7), use-after-free (CWE-416, 13.7), ~~CWE-399, and CWE-665, as well as integer overflow CWE-190, all incorrectly mapped to CWE-119, indicates that models exhibit a strong bias toward buffer overflow as a general-purpose prediction when encountering code complexity~~and several resource-management labels. LFD shows a different pattern: CWE-416 (Use After Free) ~~dominates confusions with 57 instances, followed by CWE-119, but also spans multiple targets (CWE-123, CWE-121, CWE-20), suggesting models struggle specifically with memory lifecycle issues and map them to various~~ is the most frequent source label in six of the eight leading confusion pairs and maps to several memory-related ~~categories.~~ predictions. These counts document concentration in the error distribution without establishing why the models choose those labels.

TABLE XIII
TOP-5 AND BOTTOM-5 CWE DETECTION PERFORMANCE (AVERAGED ACROSS ALL MODELS AND THREE RUNS).

PBD Benchmark		LFD Benchmark	
CWE ID	Acc. (%)	CWE ID	Acc. (%)
Top Performers		*Top Performers*	
CWE-119 (↑1)	~~57.1~~ 44.8	CWE-20 (↑1)	~~35.7~~ 34.5
CWE-20 (↑2)	~~44.8~~ 34.2	CWE-787 (↑2)	~~31.0~~ 33.3
CWE-122 (↑3)	~~40.8~~ 34.0	CWE-129 (↑3)	~~28.6~~ 29.5
~~CWE-190~~ CWE-129 (↑4)	~~31.0~~ 21.9	CWE-125 (↑4)	~~25.6~~ 25.5
~~CWE-120~~ CWE-190 (↑5)	~~23.8~~ 21.4	CWE-190 (↑5)	20.0
Bottom Performers		*Bottom Performers*	
~~CWE-327~~ CWE-191 (↓)	0.0	CWE-476 (↓)	~~12.3~~ 14.5
~~CWE-16~~ CWE-276 (↓)	0.0	CWE-835 (↓)	~~4.8~~ 3.2
~~CWE-732~~ CWE-665 (↓)	0.0	CWE-667 (↓)	2.4
~~CWE-1284~~ CWE-755 (↓)	0.0	CWE-908 (↓)	0.0
~~CWE-326~~ CWE-763 (↓)	0.0	CWE-401 (↓)	0.0

TABLE XIV
TOP-8 MISCLASSIFICATION PATTERNS SHOWING CWE-119 AS A SEMANTIC MAGNET IN PBD AND CWE-416 CONFUSION DOMINANCE IN LFD. COUNTS ARE AVERAGED ACROSS THREE RUNS.

PBD			LFD		
True	Predicted	Count	True	Predicted	Count
CWE-122	CWE-119	17.7	CWE-416	CWE-119	59.7
CWE-276	CWE-119	15.3	CWE-125	CWE-119	31.7
CWE-120	CWE-119	13.7	CWE-476	CWE-119	23.0
CWE-416	CWE-119	13.7	CWE-416	CWE-121	20.0
CWE-772	CWE-119	13.7	CWE-416	CWE-123	19.7
CWE-284	CWE-119	12.7	CWE-416	CWE-20	17.7
CWE-399	CWE-119	12.0	CWE-416	CWE-125	12.7
CWE-665	CWE-119	12.0	CWE-416	CWE-1234	12.3

HPS Validation. The HPS metric assumes that ~~hierarchical distance in the CWE-1000 taxonomy reflects~~ taxonomy relations provide a useful proxy for semantic similarity between vulnerability types. To ~~assess this assumption~~examine that proxy, we analyzed 2,~~159~~ 107 Top-1 CWE predictions

across all models (Table XV). ~~We found that only 13.8% (294~~Only 13.9% (289/2,~~132~~077) of incorrect predictions were hierarchically close (distance $\leq$3), ~~suggesting~~ so most wrong predictions are ~~taxonomically distant from the true CWE~~ not near the true label in the hierarchy (Table XVIII). For example, ~~when the true CWE is~~ a true CWE-119 ~~(Buffer Errors), StarCoder2 may predict~~ with a predicted CWE-121 ~~(Stack Overflow) with~~ has hierarchy distance 2, ~~demonstrating partial understanding of the buffer-related vulnerability family (Table ??)~~recording a buffer-family near miss without proving semantic reasoning. However, we observed systematic bias toward predicting CWE-119: this general category appears in ~~45.1~~44.2% of all top-5 predictions, with Llama reaching ~~98.1~~98.6% (Table XVI). ~~This bias inflates HPS scores without demonstrating specific vulnerability knowledge: models hedge by predicting high-abstraction parent categories that match many children~~Frequent inclusion of this broad category can raise HPS without improving exact identification. Additionally, the CWE hierarchy does not always reflect semantic similarity: CWE-129 (Array Index) and CWE-119 (Buffer Errors) are both ~~bounds violations~~ bounds-related but have distance 7~~(Table ??), meaning semantically reasonable confusions~~, so a plausible confusion can receive low HPS credit. The large ~~gap between HPS and Top-1 accuracy across models~~ HPS-to-Top-1 ratios (Table XVII) ~~, ranging from 5.9× (GPT-4.1-mini) to 15.7× (Llama) in PBD, suggests pattern-matching on taxonomy structure rather than genuine~~ can arise from repeated broad or related candidates and must not be interpreted as proof of semantic understanding. We ~~interpret HPS as measuring *taxonomy-aware prediction* rather than deep semantic comprehension, and recommend future~~ therefore interpret HPS narrowly as taxonomic proximity and recommend validation through human similarity ratings ~~correlating hierarchy distancewith expert judgments~~correlated with hierarchy distance.

V. Discussion

~~The conclusions in this section~~ Our conclusions apply specifically to off-the-shelf, pre-trained LLMs ~~evaluated~~ in a zero-shot setting ~~using standardized prompts. We~~ with standardized prompts; we do not assess ~~the effects of~~ task-specific fine-tuning, few-shot prompting, chain-of-thought reasoning, external tools, or hybrid ~~analysis pipelines. Accordingly, our findings should not be interpreted~~ pipelines. They should thus be read not as fundamental limitations of ~~large language models~~ LLMs as a class, but ~~rather~~ as evidence that current baseline ~~LLM~~ usage is insufficient for reliable ~~software~~ vulnerability detection in high-assurance settings. ~~Note that model~~ Model behavior is described ~~using observational terminology grounded in empirical outputs. References~~ observationally; references to tendencies, biases, or memorization ~~are intended to~~ characterize consistent prediction patterns~~rather than to imply internal model intent or underlying~~ , not internal intent or mechanisms.

Key Findings. Our benchmark exposes ~~fundamental limitations in LLMs' security reasoning. The low CWE detection~~ substantial limits in zero-shot LLMs vulnerability analysis. Low CWE accuracy (best ~~: 11.4% Top-5) and systematic bias toward memory-safety vulnerabilities reveal surface-level pattern recognition rather than deep security understanding. The reverse bias, where models detect patched code 13.5% better than vulnerable code, contradicts assumptions about security-conscious defaults. Performance degradation with explicit CVE/CWE context suggests additional information overwhelms rather than guides reasoning. These findings indicate that LLMs should serve~~ Top-1 only 3.3% on PBD and 15.6% on LFD), strong class biases, and frequent identical verdicts for vulnerable–patched pairs are inconsistent with reliable code-sensitive classification. Providing CWE context does not consistently help and sometimes hurts. These observations support using the evaluated models as assistive tools under expert supervision ~~, not as autonomous analyzers~~rather than autonomous analyzers; they do not by themselves identify the models' internal reasoning mechanism.

Deployment cost at multi-file context. Median Level 3 prompts are $\approx 2.7\times$ longer than Level 1 across all seven models. In the two released GPT-4.1-mini temperature-0.1 logs, the median sequential completion interval increases only slightly from 0.51 s at Level 1 to 0.53 s at Level 3 for the non-targeted PBD task; these timestamps include API and local orchestration overhead and are not a controlled service-level benchmark. For a self-hosted Llama3.1-8B on L40S, the artifact-based projection rises from 2.5 s to 6.8 s. At the pricing snapshot used in our analysis[1,2], a typical eight-output-token GPT-4.1-mini request costs approximately $0.53 per 1,~~and that significant architectural improvements are required for reliable vulnerability identification~~000 Level 3 predictions; $1.34 is a conservative upper bound if every response reaches the 512-token cap. The corresponding Llama3.1-8B estimate is $1.61, so CI/CD operators should reserve multi-file context for diff-triggered review batches rather than indiscriminate continuous scanning.

Future Directions. Future work should validate HPS through human expert studies ~~. Specifically, security practitioners should rate the semantic similarity of CWE pairs on a 1-5 scale , enabling the calculation of the correlation between hierarchy distance and perceived similarity (~~(e.g., having practitioners rate CWE-pair similarity on a 1–5 scale and correlating it with hierarchy distance via Spearman's ρ). LLMKernelBench ~~enables several research paths~~further enables: (1) analyzing LLMs' reasoning ~~processes to identify security comprehension~~ to locate security-comprehension weaknesses [**?**], [**?**]; (2) expanding to ~~multiple~~ other languages (Python, Java, Rust, Go)~~to assess cross-language transfer~~; (3) ~~investigating~~ hybrid LLM-agent architectures integrating static/dynamic analysis~~tools; ~~; and (4) examining whether LLMs introduce vulnerabilities when generating patches. ~~These directions aim to transform our diagnostic framework into a catalyst for developing robust, security-aware models.~~

[1] GPT-4.1-mini pricing at launch: https://openai.com/index/gpt-4-1/, retrieved 2026-05-06.

[2] RunPod community-cloud GPU pricing: https://www.runpod.io/pricing, retrieved 2026-05-06.

TABLE XV
~~TOP-8 MISCLASSIFICATION PATTERNS SHOWING CWE-119 AS A SEMANTIC MAGNET IN PBD AND CWE-416 CONFUSION DOMINANCE IN LFD~~HIERARCHY DISTANCE ANALYSIS FOR INCORRECT CWE PREDICTIONS ACROSS MODELS, AVERAGED ACROSS THREE RUNS. "CLOSE≤3" INDICATES PREDICTIONS WITHIN HIERARCHY DISTANCE 3 OF THE TRUE CWE. STATISTICS COMPUTED OVER N PREDICTIONS PER MODEL. PER-MODEL COUNTS ARE INDEPENDENTLY ROUNDED FROM THREE-RUN AVERAGES, SO DISPLAYED ROWS NEED NOT SUM EXACTLY TO THE OVERALL ROW.

Model	N	Wrong	Close≤3	Close (%)	Avg Dist
GPT-4.1-mini	306	296	63	21.4	4.57
Starcoder	308	304	59	19.3	4.47
Llama	312	308	56	18.3	4.53
Mistral	310	306	31	10.1	4.78
Codellama	311	309	30	9.8	5.83
Qwen3-Coder	310	303	27	9.0	5.89
Deepseek	250	249	22	8.7	4.96
Overall	2107	2077	289	13.9%	5.00

TABLE XVI
FREQUENCY OF CWE-119 APPEARING IN TOP-5 PREDICTIONS ACROSS MODELS, INDICATING SYSTEMATIC BIAS TOWARD THIS GENERAL BUFFER ERRORS CATEGORY. COUNTS AVERAGED ACROSS THREE RUNS.

Model	CWE-119 in Top-5	Total Pred.	Rate (%)
Llama	308	312	98.6
Starcoder	249	308	80.8
GPT-4.1-mini	215	306	70.3
Qwen3-Coder	99	310	31.8
Deepseek	35	250	14.1
Mistral	21	310	6.7
Codellama	6	311	1.8
Overall	932	2107	44.2

TABLE XVII
GAP BETWEEN HPS AND TOP-1 ACCURACY SHOWING POTENTIAL TAXONOMY PATTERN-MATCHING. RATIO INDICATES HOW MANY TIMES LARGER HPS IS COMPARED TO TOP-1 AND IS COMPUTED FROM UNROUNDED THREE-RUN MEANS. DATA FROM TABLE XI.

Model	PBD			LFD		
	Top-1 (%)	HPS (%)	Ratio (×)	Top-1 (%)	HPS (%)	Ratio (×)
Starcoder	1.3	19.0	14.6	0.0	26.9	∞
Deepseek	0.3	6.3	17.2	0.0	10.5	∞
Codellama	0.5	6.5	12.2	1.6	8.9	5.4
Llama	1.2	10.7	9.1	1.3	15.9	12.2
Mistral	1.1	9.1	8.5	5.2	10.7	2.0
Qwen3-Coder	2.2	15.4	6.8	6.2	21.9	3.5
GPT-4.1-mini	3.3	19.3	5.9	15.6	31.1	2.0

VI. THREATS TO VALIDITY

Although the methodology and experiments in this study were carefully designed and executed, some potential threats have to be discussed.

Internal Validity. ~~One possible threat arises from the~~ The non-deterministic nature of LLMs ~~. To minimize this effect, we set the temperature parameter to~~ is a potential threat; the primary open-source evaluation uses temperature 0 ~~in all experiments (as described in Section III)~~for deterministic decoding (§III). To assess sensitivity to stochastic decoding, we also analyze two complete temperature-0.1 runs and a temperature-1.0 run; GPT-4.1-mini is available only in the temperature-0.1 runs. Table XIX shows that response validity can change substantially with decoding, especially for StarCoder2. Prompt formulation can also affect outcomes, ~~ensuring outputs were as deterministic as possible and allowing reproducible results.~~

~~Another potential source of bias lies in the use of prompting strategies, as different formulations can yield different outcomes. In this study, we focused on assessing fundamental SVD capabilities using~~ so we use standardized zero-shot prompts ~~instead of~~ rather than optimized prompt engineering~~. This approach establishes~~, establishing a consistent and fair baseline~~for future research.~~

~~A further concern is that some~~ . Some models may have encountered PBD data during pretraining~~, which could affect their performance. To mitigate this~~; to estimate this effect, we introduced the LFD dataset ~~, composed~~ of vulnerabilities disclosed after the models~~' training cutoffs . By comparing model~~ ' training cutoffs and compare behavior across both ~~datasets, we can estimate the extent to which exposure to the training data may have affected the results~~splits.

~~Another threat is that manual annotation was used to build the dataset, introducing an element of subjectivity. To mitigate this, clear annotation guidelineswere established, and both reviewers adhered strictly to them. In cases of uncertainty, the reviewers discussed the instance, thereby reducing individual bias and ensuring consistent labeling.~~

Manual annotation introduces subjectivity, which we mitigated with clear guidelines, strict reviewer adherence, and consensus discussion of uncertain cases. Extracting only modified functions may ~~lack the surrounding contextfor vulnerability detection. We mitigate this by including~~ omit surrounding context; we address this through three abstraction levels~~. Our~~ , validated by the distinct performance patterns in our RQ3 analysis (§IV-C)~~validates this design by demonstrating distinct performance patterns across these context levels.~~

. Finally, all ~~evaluated~~ tasks focus on static code reasoning ~~. As a result, they do not account for dynamic execution behavior~~ and do not capture dynamic execution or runtime verification~~. While this design~~ , which isolates reasoning capability ~~from execution effects, it may limit the interpretation of results in contexts~~ but may limit interpretation in settings requiring dynamic analysis.

External Validity. Our analysis is restricted to Linux

TABLE XVIII
DISTRIBUTION OF HIERARCHY DISTANCES FOR INCORRECT CWE PREDICTIONS WITH MEASURABLE DISTANCES ($N \approx 1708$ OF ≈2077 INCORRECT PREDICTIONS PER RUN ON AVERAGE; COUNTS AVERAGED ACROSS THREE RUNS). 16.9% OF THESE ERRORS FALL WITHIN PROXIMITY ≤ 3 (13.9% OF ALL INCORRECT PREDICTIONS).

Distance	Count	Prop. (%)	Cumul. (%)
1	46.0	2.7	2.7
2	123.3	7.2	9.9
3	119.3	7.0	16.9
4	316.7	18.5	35.4
5	421.7	24.7	60.1
6	365.3	21.4	81.5
7	219.0	12.8	94.4
8	72.7	4.3	98.6
9	23.7	1.4	100.0
Total ≤ 3	**288.7**	**16.9**	–

kernel C code, ~~raising questions about its~~ which may limit generalizability to other ~~programming~~ languages or domains~~. Nonetheless, the Linux kernel'~~; nonetheless, the kernel's scale, complexity, and ~~role in~~ security-critical ~~systems~~ role make it a meaningful ~~and representative benchmark for real-world challenges in understanding code.~~

~~Additionally, our evaluation does not include the most recent state-of-the-art models, such as GPT-5. However, the study includes a diverse set of seven models varying in architecture and size. The consistency of observed trends across these models, along with their alignment with the behavior of larger proprietary models such as GPT-4.1-mini, gives us confidence that our findings extend beyond the evaluated models~~stress-test domain. Models released after our evaluation are not included, and the observed consistency across the seven evaluated systems does not guarantee transfer to newer architectures or other software domains.

Construct Validity. HPS relies on a manually defined weighting scheme to quantify hierarchical proximity within the CWE taxonomy. The ~~metric is intended to preserve relative semantic ordering across hierarchy levels rather than depend on specific numeric values; accordingly, our analysis focuses on comparative trends and model rankings rather than~~ weights encode an ordinal taxonomic ordering rather than calibrated semantic distances, so absolute HPS magnitudes should be interpreted cautiously. We do not perform a sensitivity analysis over alternative weights; model rankings and gaps could change under another defensible scheme. ~~Variations in weightsthat preserve hierarchy order are unlikely to affect the qualitative conclusions.~~

CWE assignment, particularly at fine-grained base and variant levels, involves inherent subjectivity even among experts. This is especially relevant for the LFD, where some CWEs were manually assigned following NVD-aligned guidelines, and formal inter-annotator agreement was not computed. Consequently, part of the low Top-1 accuracy in fine-grained CWE classification may reflect label ambiguity rather than purely model failure. To mitigate this, we emphasize hierarchy-aware evaluation and relative performance trends, which are more robust to label noise.

Conclusion Validity. Our conclusions are based on pre-trained LLMs evaluated in a zero-shot setting with standardized prompts. While this enables fair and reproducible comparison, it does not capture the effects of prompt adaptation, few-shot examples, or task-specific fine-tuning. Thus, the observed limitations reflect the reliability of baseline LLM deployment rather than definitive bounds on LLM security reasoning.

Although appropriate statistical tests and effect sizes are reported, the LFD's limited size constrains statistical power, particularly at higher abstraction levels. As a result, some differences may be sensitive to sampling variance, and non-significant results should not be interpreted as evidence of equivalence. The LFD is intended to reveal robustness trends under leakage-free conditions rather than support fine-grained statistical generalization. Likewise, per-CWE estimates for classes with fewer than approximately 20 samples are exploratory and should not support statistically definitive class-level claims. Finally, descriptions of model behavior (e.g., bias or memorization) refer to observed prediction patterns and do not imply internal model intent or mechanisms.

VII. Conclusion

This paper presents LLMKernelBench, a benchmark for evaluating LLM capabilities in SVD and CWE classification using 417 real-world Linux kernel vulnerabilities. Our evaluation of seven models reveals ~~fundamental~~ substantial limitations: binary ~~vulnerability~~ detection achieves near-random performance (∼50% accuracy), CWE classification remains weak (best: ~~14.7~~15.6% Top-1, ~~51.0~~52.7% Top-5), and models exhibit strong ~~bias toward memory-safety vulnerabilities. Abstraction level~~ prediction biases. Abstraction analysis shows model-specific responses~~(some improve with added context,~~ , with some improving on added context while others degrade by up to ~~22%). The leakage-free dataset (LFD) reveals significant performance gaps:~~ 13.4%. On LFD, abstraction robustness is model-specific rather than tied to specialization (code-specialized ~~models degrade by 8.8% , while~~ 2.6% vs. general-purpose ~~models demonstrate superior robustness with minimal degradation(0.3%) . These findings demonstrate that current LLMs lack robust security reasoning and~~ 4.5% average degradation), while the larger micro-to-macro CWE-accuracy gap (0.9 points in PBD and 6.6 in LFD) is consistent with class-frequency sensitivity but may also reflect split composition. The evaluated zero-shot models therefore require expert supervision~~.~~ , and LLMKernelBench provides a reproducible foundation for ~~developing more reliable vulnerability detection tools~~building more reliable detectors.

Appendix: Decoding Temperature and Output Validity (Added)

We audit raw SVD responses from deterministic decoding (T = 0), two independent T = 0.1 runs, and T = 1.0. A genuine abstention clearly selects `not sure`, including a verbose response only when its final intent is unambiguously uncertainty due to insufficient information. Invalid responses include code continuation, malformed or looping text, contradictory choices, and outputs with no resolvable verdict. Table XIX reports both rates as percentages of all responses, pooled over non-targeted/targeted settings and PBD/LFD; the T = 0.1 column is the arithmetic mean of the two runs.

TABLE XIX
Genuine abstention and invalid-response rates (% of all responses), shown as genuine/invalid. Rates are pooled over both SVD settings and both datasets; T = 0.1 is averaged across two runs. GPT-4.1-mini was available only at T = 0.1.

Model	T=0	T=0.1 (two-run avg.)	T=1.0
CodeLlama	0.00/1.80	0.00/1.83	2.34/1.20
DeepSeek-R1	0.00/0.00	0.00/0.00	0.00/0.00
GPT-4.1-mini	–	2.40/0.00	–
Llama3.1	0.00/0.00	0.00/0.00	0.00/0.00
Mistral	0.00/1.38	0.00/1.47	0.48/0.18
Qwen3-Coder	0.06/0.48	0.12/0.57	1.80/0.18
StarCoder2	0.00/26.08	0.00/22.45	1.14/1.92

The corrected audit does not support a universal 0% abstention claim. Genuine abstention is still uncommon but

averages 2.40% for GPT-4.1-mini at T = 0.1, reaches 2.34% for CodeLlama at T = 1.0, and reaches 1.80% for Qwen3-Coder at T = 1.0. Invalid output is a separate and often larger failure mode. It is concentrated in StarCoder2 (26.08% at T = 0 and 22.45% averaged across the two T = 0.1 runs), then drops to 1.92% at T = 1.0. Thus, most StarCoder2 non-answers reflect prompt-format non-compliance rather than epistemic uncertainty, whereas GPT-4.1-mini's observed non-binary responses are genuine abstentions.